Fuzix for the Pico Computer

Unix and BBC BASIC on the Pico Computer 2 and 3

Release v0.17 — August 2026

Contents

1 Introduction

Fuzix is Alan Cox's compact V7-style Unix. This port makes it a third first-class environment for the Pico Computer, beside MMBasic and MicroPython: a genuine multi-tasking Unix booting from SD card, with a full set of classic utilities — and R. T. Russell's BBC BASIC as its flagship application, complete with BBC graphics modes on the HDMI display, the four-channel BBC sound system on the audio DAC, joystick and analogue inputs, and a spare serial port.

It also compiles on its own. `cc` is a complete C89 toolchain running on the hardware — not a cross-compiler — and it generates native ARM code, so the machine builds its own programs, at its own speed, with no PC involved. `mmbc` puts MMBasic in front of it: an MMBasic program translates to C, compiles, and runs as a native program several times faster than the interpreter it was written for.

One kernel image serves both machines. At boot it probes GP27 for the DS3231's 32 kHz clock (the same test MMBasic and MicroPython use): present means Pico Computer 3, absent means Pico Computer 2. The practical difference is the SD card wiring, handled automatically; the boot banner names the detected board.

Headline specification as configured here:

- CPU at 375 MHz (the XGA-capable clock, as MMBasic uses)
- 340 KB of program RAM managed as 4 KB pages, with the 8 MB PSRAM behind it — up to 64 concurrent processes, and a single process may have about 292 KB of it. A swapped-out process is a PSRAM allocation the size of the process, not a slot on a device, so nothing is reserved for one that does not exist
- Programs get their heap from the PSRAM too, so a BASIC array or a C `malloc` is limited by the 8 MB rather than by the process
- Root filesystem on SD card; the on-board NAND flash holds a recovery system
- 80×40 colour ANSI console on HDMI, mirrored to the USB-C serial port, with USB keyboard support (six layouts)
- Pre-emptive multitasking: a runaway program can always be stopped from the keyboard
- A self-hosted C89 compiler generating native ARM code, and an MMBasic translator in front of it — both run on the machine itself
- MMBasic's own full-screen editor, `mmedit`, so BASIC is written, translated, compiled and run without leaving the machine

1.1 New in v0.17

The machine keeps its keyboard, and BASIC finishes a category.

A program that only printed could not be stopped from the USB keyboard. `Ctrl-C` from a serial terminal worked; from the keyboard it was ignored. The keystroke was never decoded rather than lost on the way to the program: the USB host stack is pumped when the kernel idles, when a reader is about to block, and when a spinning process is pre-empted — and a process that only *writes* reaches none of the three, because it is always runnable, never reads the tty, and is inside `write()` for almost the whole of a line. There is now a fourth pump on the output path. The symptom was worth recording: typing during such a program produced exactly one character, the first key pressed, and only once the program had ended.

`LOAD JPG`, `LOAD PNG` and `SPRITE LOADPNG`. MMBasic's own picojpeg and upng, as separate programs, so a BASIC program that never touches an image carries no decoder. Note the transparency default: it is 0, which is palette black, so a PNG with a transparent surround draws a black box unless you pass **-1**. A real PicoMite does the same. See Running another program and Commands at the # prompt.

ARRAY SLICE and ARRAY INSERT — one line through an array of two or more dimensions, far quicker than the equivalent FOR loop, and spelled MATH SLICE/MATH INSERT as well because the interpreter runs both through the same two functions. **COLOUR MAP** turns a whole array of colour codes into RGB888 in one statement. That empties the "finish what is already there" category: 220 of MMBasic's 353 names now translate.

DIM s(4) = (1,2,3,4) under OPTION BASE 1 filled the wrong elements — s(1) held 2 and s(4) held nothing. Silent wrong answers, in a shape no program would suspect.

A refused batch of pixels is an error. LOAD IMAGE, LOAD JPG and LOAD PNG ignored what the kernel told them, so a picture could be quietly incomplete — up to 256 pixels of it. They stop now.

mmedit's function-key line was always one keystroke late: blank on entering the editor until you pressed something, or for the five seconds it took the status line to redraw itself. Measured 5.43 s before and 0.43 s after.

This chapter is now the only one of its kind. Everything the older "New in" sections described that is still true of the machine has been moved into the section that documents it — the pin timings into the pin chapter, PAUSE's behaviour into the timer chapter, the recursion limit into the chapter on making a program fit, and so on. A release note is a poor place to look something up.

ewpage

2 Installing Fuzix

2.1 Flashing the kernel

The kernel is a standard UF2 file, fuzix.uf2, flashed exactly like any other Pico Computer firmware:

1. Set the USB HUB switch to **DISABLE** and connect the **Prog** port to your PC.
2. Hold **BOOT**, click **RESET**, release BOOT. A USB drive appears.
3. Drag fuzix.uf2 onto the drive. The board reboots when the copy completes.
4. Set the HUB switch back to **ENABLE**.

Flashing does not touch the SD card.

2.2 Writing the SD card

Fuzix boots from a ready-made card image, pc3-sd.img (978 MB, about 4.9 MB compressed for download). Write it to a card of **1 GB or larger** with any raw-image tool — Raspberry Pi Imager ("use custom image"), Win32DiskImager, balenaEtcher, or dd on Linux/macOS. **The whole card is overwritten.**

The image lays the card out as three partitions:

Partition	Size	Type	Purpose
1	128 MB	FAT	File interchange with Windows/macOS/Linux
2	800 MB	Fuzix root	The Unix filesystem (boot device hdb2)
3	4 MB	0x7F	Reserved

The root filesystem is 1,638,400 blocks of 512 bytes with 25,600 inodes, and it fills its partition exactly.

It did not always. Before v0.9 a block number was 16 bits, so 65535 was at once the highest block a filesystem could have and the marker meaning "no such block" — and the filesystem was deliberately held to 64000 blocks, short of its own partition, so that a stray value could be recognised as wrong instead of pointing at a real sector. That is what limited the root to 32 MB. From v0.9 the format is **FS32**: 32-bit block numbers and 256-byte inodes, with a "no such block" marker that cannot be a real block at all. The margin is gone, and with it the ceiling — a filesystem can now be up to 2 TB and a single file up to 1 GB.

A v0.9 kernel and a v0.9 card go together. The two formats are not interchangeable and neither pretends otherwise: a v0.9 kernel refuses an older card by name at the `bootdev:` prompt, and an older kernel refuses a v0.9 one. Flash `fuzix.uf2` and write the card in the same sitting.

Since v0.5 the filesystem has been built from source by `mksdimage.sh`, so the card can be reproduced rather than merely copied. `mkcard.sh` decides the layout, and its `FAT_MB` and `ROOT_MB` settings are the only place the partition sizes are written down.

Partition 1 ships unformatted: format it as **FAT or FAT32 (not exFAT)** in Windows before first use — see section 6. Partition 2 is Fuzix's own filesystem format; no desktop OS can mount it, which is why the FAT partition and the `fat` command exist.

2.3 First boot

Connect a monitor to the HDMI port and/or a terminal program (for example TeraTerm at 115200) to the USB-C console port. Switch on: the kernel banner, board identification and device probe appear on both, then:

```
login: root
```

There is no password. The system arrives at a Bourne shell; the console is an 80×40 colour terminal (termcap type `pc3`) and the USB keyboard and serial input feed the same session interchangeably. Set the keyboard layout in `/etc/rc` (`picoctl keymap uk` by default).

Auto-repeat waits 250 ms and then repeats every 50 ms — twenty a second. HID keyboards only report a change of state, so the repeat is synthesised from the held key. These were MMBasic's 600/150, which is a typewriter's rate and fine for typing, but anything *held* suffers: a game played from the keyboard crawled beside the same game played from a switch array, which is read fresh every pass of its loop with no typematic delay at all. The standard `KBRATE` tty ioctl adjusts them at run time, but note that its units are TENTHS of a second and so cannot express the 50 ms default.

2.3.1 Num lock, and keyboards with no keypad

A keyboard with no numeric keypad — a Raspberry Pi keyboard, and most laptop-style boards — usually overlays one onto `7890/uiop/jkl;/m` whenever num lock is on, so those letters type digits. The keyboard's own firmware does that, in response to the num lock light the machine asks it to show.

Two things handle it. A keyboard that reports **no num lock light** is taken to have no keypad and starts with num lock off (it says so at boot). That does not catch every such keyboard — one that reports a num lock light while having no keypad is indistinguishable from a full-size keyboard — so **pressing Num Lock is also remembered**, per keyboard.

To ask what is going on:

```
# picoctl numlock
num lock on
```

```
keyboard 04d9:0006, num lock LED declared
```

and to change it: `picoctl numlock off`. **That is remembered across reboots**: the kernel itself writes no files, so `picoctl` keeps a line per keyboard in `/etc/numlock` whenever a setting changes, and `/etc/rc` replays them with `picoctl numlock --load` at every boot. Add `--once` to change the setting without saving it. The full syntax, including setting a keyboard that is not plugged in yet, is in Commands at the # prompt.

One thing that can mislead: the keyboard remembers its own num-lock state, and the hub is externally powered, so it keeps that state across a warm reboot. A reading taken right after a reboot tells you what the keyboard is doing, not what the machine decided.

To power off, type `shutdown` (or at minimum `sync`, then wait a moment). After an unclean power-off the next boot repairs the filesystem automatically (`fsck -a -y`).

2.4 Setting the clock

The board's DS3231 battery-backed clock supplies the date and time: at every boot `/etc/rc` runs `setdate`, which reads the chip and sets system time silently. If the chip cannot supply a valid time (fresh battery, or the battery was removed) the same command falls back to prompting on the console:

```
Current date is Mon 2026-07-27
Enter new date:
```

Press Enter to keep a value, or type a new one (`2026-07-28`, then `16:30:00`). To set the clock at any other time run `setdate -u`, which always prompts.

Setting the system time does not touch the chip. To make the time permanent, write it back:

```
# setdate -w
writing
```

That also restarts a stopped oscillator, so the sequence for a new battery is: `setdate -u` (enter the time), then `setdate -w`. The kernel message `oscillator was stopped, time needs setting` at boot means exactly that sequence is wanted.

While running, system time is kept by the crystal-driven timer tick and gently re-synchronised from the DS3231 about once an hour, the same policy as MMBasic. `date` prints the current time.

If a transaction to the chip is ever interrupted at the wrong moment (a reset mid-read), the battery-backed DS3231 can be left jamming the I2C bus - even across power cycles. The kernel detects this at boot (`ds3231: SDA held low, clocking bus free`) and clears it automatically.

2.5 Command history and line editing

At any cooked-mode prompt - the shell, login, most line-oriented programs - the console provides in-line editing and command history:

Key	Action
Up / Down	walk back / forward through previous commands
Left / Right	move within the line
Home / End (or Ctrl-A / Ctrl-E)	start / end of line
Backspace / Delete	delete left / at the cursor
Ctrl-U	discard the line

The history (about 500 commands) lives in a small region reserved at the top of the PSRAM, so it costs no program memory; like the RAM disc it survives a reset but not a power cycle. Programs that take the terminal raw (BBC BASIC, the editor) see every keystroke themselves as usual - BBC BASIC has its own line editor and *EDIT.

2.6 Escape routes

- **Esc** — inside BBC BASIC, stops the running program.
- **Ctrl-** — kernel emergency break: kills the foreground program even if it has the terminal in raw mode, and restores a sane terminal. Use it instead of the RESET button when something wedges.
- `kill` from the shell works on anything; Fuzix pre-emption guarantees a spinning process can't lock you out.

3 The consoles

Everything is mirrored: kernel and program output appear on the HDMI display and the serial console simultaneously, and input is merged from the USB keyboard and the serial line. TeraTerm is resized to 80×40 automatically at boot.

While BBC BASIC has a graphics MODE on screen, the HDMI display belongs to the graphics; text output continues to the serial side as a plain stream, so a transcript survives even a full-screen game.

4 BBC BASIC

Start it by name; leave with `QUIT`:

```
# bbcbasic
BBC BASIC for Fuzix Console v0.50
(C) Copyright R. T. Russell, 2025
>
```

This is R. T. Russell's BBC BASIC (the BBCSDL console edition), so the language is the full modern dialect: 64-bit integers, IEEE double floats, long variable names, `WHILE`/`ENDWHILE`, multi-line `IF`/`ELSE`/`ENDIF`, structures, and the inline assembler. Keywords must be typed in **capitals** (`PRINT`, not `print`); `*LOWERCASE ON` relaxes this. Star commands are case-insensitive.

4.1 Loading programs, including plain text

`LOAD` and `CHAIN` accept **both** program formats:

- the tokenised internal format written by `SAVE` (fast, exact), and
- **plain ASCII listings**, detected automatically. An unnumbered modern-style listing is given line numbers 10, 20, 30…; a listing with its own numbers keeps them. Windows line endings are fine, and typographic characters that desktop editors introduce (curly quotes, non-breaking spaces, tabs) are silently converted to their ASCII equivalents — an invisible "smart" character can otherwise produce a baffling `Syntax error` on a line that looks perfect.

The comfortable round trip: edit `myprog.bas` on your PC, copy it to the FAT partition, then:

```
# fat get myprog.bas
# bbcbasic
>LOAD "myprog.bas"
>RUN
>SAVE "MYPROG"          save tokenised for fast loading next time
```

4.2 Graphics

`MODE 0`–`5` switch the display to 1024×768 and provide the classic screens; `MODE` with any higher number (e.g. `MODE 6`) returns to the text console. Errors leave you at the prompt *in* the current mode, exactly like the original machine.

MODE	Resolution	Colours	Presentation on the monitor
0, 3	640×256	2	Full width (5:8 smooth upscale), full height
1, 4	320×256	4	960×768 — every pixel a crisp 3×3 block
2, 5	160×256	16	960×768 — every pixel a crisp 6×3 block

Coordinates are the authentic 1280×1024 graphics units, origin bottom-left, movable with `VDU 29`. Implemented: `MOVE`, `DRAW`, `PLOT` (move/line families 0–15, points 64–71, filled triangles 80–87 — so `PLOT 85` works), `CLG`, `GCOL`, `COLOUR`, `POINT(x,y)`, `VDU 19` palette changes (instant, no redraw), `TAB(x,y)`, and text printed with the built-in 8×8 font (40 or 80 columns × 32 rows, with scrolling). `MODE 1/4` store 4 bits per pixel, so all 16 logical colours are available there via `VDU 19` even though the default palette is the authentic four.

4.3 Sound

The full BBC sound system plays through the PCM5102 DAC and the 3.5 mm jack:

```
SOUND 1,-15,89,20                              A4 (440 Hz) for one second
ENVELOPE 1,1,4,-4,4,10,10,10,126,-2,0,-10,120,100
SOUND 1,1,100,30                               envelope-shaped note
SOUND &201,-15,53,20:SOUND &202,-15,69,20:SOUND &203,-15,81,20
                                               a synchronised C-E-G chord
```

Channels 1–3 are square-wave tones, channel 0 is noise. Each channel queues up to 8 notes; `&1x` in the channel number flushes the queue. `&Sxx` holds the note until S *other* channels carry the same sync mark, so a three-note chord wants `&2xx` on each channel — `&1xx` releases in pairs, playing two notes and leaving the third waiting for a partner that never comes. `ENVELOPE` implements the three-section pitch envelope and ADSR amplitude, stepped at the authentic 100 Hz. Pitch follows the BBC scale: 4 units per semitone, 89 = A4 = 440 Hz. Durations are in 20ths of a second; 255 means "until further notice". `SOUND` blocks BBC-style when a queue is full (Esc still works).

A note on character. The output is hot: one full-amplitude (`-15`) square is a quarter of DAC full scale and three sum to nearly three-quarters, which can clip a sensitive line input that handles MP3 playback happily — drop to `-7` if the input distorts. The squares themselves are band-limited (polyBLEP), so sustained notes hold a pure pitch right to the top of the range instead of carrying the aliasing shimmer a naive digital square picks up.

There is one audio output, so the synthesiser and MP3 playback are mutually exclusive — as they are in MMBasic. While a track is playing, notes are queued and heard when it ends; `PLAY STOP`, or `killing playmp3`, gives the synthesiser the hardware back at once.

4.4 ADVAL: joystick, analogue inputs, sound queues

Call	Returns
`ADVAL(0)`	Joystick switches on GP34–37 as a mask: bit 0 = GP34, bit 1 = GP35, bit 2 = GP36, bit 3 = GP37; pressed (grounded) = 1
`ADVAL(1)`–`ADVAL(4)`	Analogue readings of GP41–GP44, 0–65520 (12-bit ADC ×16)
`ADVAL(-1)`	Characters waiting in the keyboard buffer
`ADVAL(-5)`–`ADVAL(-8)`	Free queue slots on sound channels 0–3
`ADVAL(-9)`	Hardware microsecond counter (64-bit)

The joystick inputs have pull-ups and Schmitt-trigger inputs enabled; wire switches directly to ground. The analogue inputs are 3.3 V full-scale.

`ADVAL(-9)` is the benchmarking clock — the RP2350's free-running microsecond timer, far finer than `TIME`'s centiseconds (which tick in tenths of a second on this kernel). MMBasic-style usage:

```
DEF FNtimer = ADVAL(-9)
T% = FNtimer
REM ... code under test ...
PRINT (FNtimer - T%) / 1000; " ms"
```

Each reading costs one system call (a few tens of microseconds) - negligible over millisecond-scale measurements. The value is the full 64-bit hardware counter (BBC BASIC integers are 64-bit), counting microseconds since power-on: it never wraps in practice.

4.5 Serial port

A second serial port lives on the I/O header: **TX = GP0, RX = GP1** (3.3 V logic, no flow control), visible as `/dev/tty2`. BBC BASIC reaches it as a *port channel* — raw and unbuffered:

```
*stty 9600 </dev/tty2
ch% = OPENUP("/dev/tty2")
BPUT#ch%,65                 send a byte
X% = BGET#ch%               receive a byte (waits for one)
CLOSE#ch%
```

Any `stty` setting applies (baud rate, parity, size). The same `OPENIN`/`OPENUP`/`BGET#`/`BPUT#` mechanism works on any `/dev` device; ordinary file names get the buffered 256-byte file channels instead, exactly as on the original machine.

4.6 Star commands and the shell

The built-in set includes `*DIR`/`*.`, `*CD`, `*TYPE`, `*COPY`, `*DELETE`/`*ERA`, `*MKDIR`, `*RMDIR`, `*KEY`, `*SPOOL`, `*EXEC`, `*LOWERCASE`, `*TEMPO`, `*QUIT`. Anything unrecognised is passed to the Unix shell, so `*ls -l`, `*stty 9600 </dev/tty2` and even `*fat get demo.bas` work from inside BASIC.

4.7 The inline assembler

The assembler between `[` and `]` targets the machine it runs on: **ARM Thumb (v6-M subset)**, not 6502. `CALL` and `USR` work as documented for BBCSDL's ARM editions. Machine code written for a BBC Micro will not run; this is the same situation as every modern BBC BASIC.

4.8 Differences from the original BBC Micro

Better than the original:

- ~110 KB of program workspace (about 80 KB when graphics are in use) against the Model B's 32 KB shared with the screen
- 64-bit integers, double-precision floats, the modern language
- Long file names on a hierarchical filesystem
- The machine multitasks underneath — BASIC is just a process

Not (yet) present, compared with an original machine or full BBCSDL:

- **MODE 7** teletext (MODE 6 and 7 return to the console)
- `VDU 5` text-at-the-graphics-cursor; user-defined characters (`VDU 23`) are accepted but not rendered
- PLOT families beyond points/lines/triangles: no circles, arcs, ellipses, flood fill or rectangle/parallelogram fills yet
- `GCOL` action modes 1–4 (OR/AND/EOR/invert) plot as mode 0 (set)
- Flashing colours (8–15 map to their steady equivalents)
- `*FX` calls, `INKEY` with negative arguments (keyboard scanning), and the 6502 `CALL` interface
- `TIME` advances in 0.1 s steps (the kernel clock granularity)
- Cassette/Econet/ROM filing systems, for obvious reasons

5 The C compiler

The machine compiles C on its own, with no PC involved. `cc` is Alan Cox's Fuzix Compiler Kit, retargeted here to a bytecode machine.

```
# cat > hello.c
#include <stdio.h>

int main(void)
{
        printf("hello from Fuzix\n");
        return 0;
}
^D
# cc hello.c
# ./hello.bc
hello from Fuzix
```

`cc prog.c` writes `prog.bc` and makes it executable, so `./prog.bc` runs it directly — the object starts with `#!/usr/bin/bcrun`, and the kernel does the rest. `bcrun prog.bc` still works and is the way to run an object that has lost its execute bit. Options are `-o name`, `-v` to show each pass as it runs, and `-k` to keep the intermediates. `bcdump prog.bc` disassembles.

`cc` is a driver: it runs `cpp`, then the three compiler passes from `/usr/lib/cc`. The passes cannot be driven from the shell by hand — two of them read back from their own standard output, which needs a redirection the Bourne shell here does not have. Like BBC BASIC, the compiler lives on the SD card root and is not in the NAND recovery system.

A small program takes about a second to compile. The dots `cc` prints are progress, one per top-level declaration.

5.1 What it compiles

C89, plus declarations after statements — the one C99 borrowing, because refusing it is a nuisance out of proportion to the standard it comes from. Everything else is ANSI C as of 1989: the full type system, `struct` and `union` including passing and returning them by value, `enum`, `typedef`, function pointers, `switch`, `goto`, all the usual operators, 64-bit `long long`, and double-precision floating point.

The compiler is checked against gcc as an oracle: 165 of the 175 applicable tests in the public `c-testsuite` conformance suite pass with byte-identical output, and a set of samples in the source tree is diffed against gcc on every change — on the board as well as on the host.

5.2 Headers and the runtime library

`/usr/lib/cc/include` holds `stdio.h`, `stdlib.h`, `string.h`, `math.h`, `assert.h`, `limits.h` and `stddef.h`. These describe what `bcrun` actually provides, and they are deliberately **not** `/usr/include`, which describes the Fuzix C library used by native binaries.

Available: `printf sprintf puts putchar`; `fopen fclose fread fwrite fgetc getc fputc putc fgets fputs fprintf feof fseek ftell rewind fflush remove`; `malloc calloc realloc free exit atoi abs`; `strlen strcpy strncpy strcat strcmp strncmp strchr strrchr memset memcpy memmove memcmp`; and from `math.h`, in double precision, `sin cos tan asin acos atan atan2 sinh cosh tanh sqrt exp`

`log log10 pow floor ceil fabs fmod`. Each of those is a native function inside `bcrun`, so it runs at machine speed whatever the calling code is.

`malloc` takes its memory from the PSRAM, not from the program's own 340 KB, so a C program can allocate far more than it could hold itself. It is asked for once when the program loads; set `BCRUN_HEAP` to a size in KB in the environment to change how much.

The raw system calls `open creat close read write lseek unlink` are also linked in, but no header declares them — declare them yourself if you want them. Two extras reach the hardware: `time_us()` returns the microsecond counter, and `adval(n)` is the BASIC `ADVAL` function (joystick, analogue inputs, sound queues).

5.3 Limitations

These are worth knowing before you start a large program.

- **One source file per program.** There is no assembler and no linker: `cc2` writes a loadable object directly, so nothing can be linked together. `cc` rejects a second source file rather than pretending.
- **Bitfields are not implemented.** `unsigned flags : 3;` is refused with a diagnostic. This is the only part of C89 missing.
- **`&` on a library function does not work.** A runtime function is resolved by index and has no address, so `&printf` cannot be represented. `bcrun` refuses such a program by name when it loads it, rather than running it with something wrong in place of an address. Pointers to *your own* functions are entirely fine.
- **No wide strings.** `L'x'` is accepted (and equals `'x'` here); `L"..."` is refused rather than quietly returned as a narrow array.
- **`printf` rounds halves up.** `%.2f` of `0.125` gives `0.13` where a full C library gives `0.12`. A deliberate, documented difference: matching the round-half-to-even rule needs exact decimal expansion.

5.4 Speed

`cc2` translates each function it compiles into native Thumb-2 for the Cortex-M33 and stores that in the object beside the bytecode; `bcrun` runs the native code and keeps the interpreter for whatever did not translate. Nothing needs to be asked for — it is simply what the compiler does.

Dhrystone 2.1, compiled on the machine, runs at about **167,000 Dhrystones/second** — 44% of the same benchmark cross-compiled by `gcc -O2` for the same chip (379,000). Individual loops do better: a sieve, a shell sort and an xorshift generator all land within a factor of two or three of gcc, and double-precision arithmetic uses the RP2350's hardware co-processor through the same routines the rest of the system uses.

That figure has moved a long way and is still moving: the same benchmark ran at 3,808/second on the original pure interpreter and 90,000 as recently as v0.8. Most of the recent ground came from keeping the hot code in native form rather than from generating better instructions — binding every library symbol once at load, caching the hottest local in a register, and inlining the compound assignments (`i += n`) on the 64-bit and double types that every MMBasic loop counter uses. Each of those had been quietly leaving native code and crossing back into the C runtime once per iteration.

A program can be forced to interpret with `BCRUN_BYTECODE=1` in the environment, which is how the native code is checked against the interpreter — the two must agree exactly.

What the compiler is *for* is being able to write and build real programs on the machine itself — utilities,

file handling, glue, and now whole applications where the speed matters. Nothing else on the Pico Computer can do it: this is a complete toolchain that runs on the hardware, not a cross-compiler.

The bytecode is also a deliberately language-neutral intermediate form, so front ends for other languages can share the same back end and runtime. `mmbc`, the MMBasic translator described next, is the first.

6 MMBasic: the mmbc translator

mmbc translates an MMBasic program into C. cc compiles that C into native ARM code. The result runs as an ordinary program, several times faster than the same source under the MMBasic interpreter, on the same machine at the same clock.

Nothing about it is a cross-development trick: both programs live on the SD card and run on the Pico Computer.

6.1 The three commands

```
# mmbc prog.bas          -> prog.c
# cc prog.c              -> prog.bc
# ./prog.bc              runs it
```

mmbc writes prog.c next to the source and says so. -o name.c puts it somewhere else. Every line it cannot translate is reported with its line number and left as a comment in the C, so a program that uses something unsupported still produces a translation you can read, and tells you exactly where it gave up.

On the machine, mmbc writes C for the machine's own compiler. On a host it writes the slightly different C that gcc prefers; --fcc and --gcc force either form, which only matters if you are moving the generated C between the two.

There is a shortcut. **cc prog.bas does both steps in one command**: it runs mmbc for you, into prog.mb.c — a deliberately different name, so that C you wrote yourself in prog.c is never overwritten — compiles that, and deletes it again with the other intermediates. It is the quickest way to get from BASIC to a running program, and the examples in this chapter are written the long way only so that each step is visible. Use mmbc on its own — or cc -k — whenever you want to keep the generated C and read it.

6.2 A first program

```
# cat > hello.bas
Print "Hello from MMBasic"
For i = 1 To 5
  Print i, i * i
Next i
^D
# mmbc hello.bas
wrote hello.c
# cc hello.c
....
# ./hello.bc
Hello from MMBasic
 1       1
 2       4
 3       9
 4       16
 5       25
```

The dots from cc are one per top-level declaration — a translated program carries the MMBasic runtime's declarations with it, so there are a couple of hundred of them even for four lines of BASIC.

That is also why the first compile takes longer than the size of your program suggests: about half a minute here, and much the same for anything small.

`Print` with a comma gives MMBasic's tabbed columns, as above.

6.3 A worked example: the KnivD benchmark

This is a published MMBasic benchmark, unmodified. It runs four thirty-second rounds and reports a score.

```
# cat bench.bas
Print "MMBASIC benchmark (C) KnivD 2016"
Dim integer t, i=0
Dim float x(1000), f=0.0
Dim string s=""
Print "Calculating... ";
count%=0
Do
  s=""
  f=0.0
  i=0
  Timer =0
  Do While Timer<30000
    i=i+2 : f=f+2.0002
    If (i Mod 2)=0 Then
      i=i*2 : i=i\2
      f=f*2.0002 : f=f/2.0002
    End If
    i=i-1
    For t=1 To 100
      f=f-1.0001
      If (f-Int(f))>=0.5 Then
        f=Sin(f*Log(i))
        s=Str$(f,6,6)
      End If
      f=(f-Tan(i))*(Rnd(0)/i)
      If Instr(s,LEFT$(Str$(i),2))>0 Then s=s+"0"
    Next
    x(1+(i Mod 1000))=f
  Loop
  Print Chr$(13)+"Performance: "+Str$((i*1024)\286,8,0)+" grains"
  Inc count%
Loop Until count%=4
End

# mmbc bench.bas
wrote bench.c
# cc bench.c
.....................................................................
# ./bench.bc
```

```
MMBASIC benchmark (C) KnivD 2016
Performance:    55353 grains
Performance:    55360 grains
Performance:    55353 grains
Performance:    55356 grains
```

MMBasic itself scores about 12,000 grains on the same board at the same clock. Note what the program exercises: 64-bit integers, doubles, string building, `Str$`, `Instr`, `Sin`, `Log`, `Tan`, `Rnd`, a thousand-element array and a timer — all of it translated, compiled and running natively.

Read the timings in this chapter as figures to compare against rather than constants. They are wall-clock measurements on a machine with other processes in it, they move a percent or two between runs, and they drift as the language grows — this one has roughly doubled since it was first printed here, and the eclipse below has slowed a little as more of MMBasic went in.

6.4 A worked example: something numerical

A longer one. It is a 3,200-line astronomical calculation (Bessel elements, lunar and solar series) that reads a date and prints the circumstances of an eclipse, and it is a good test because every digit of its output can be checked against other machines. It is among the examples as `eclipse.bas`, and in `/root/cc` as `solar_eclipse.bas` with a `solar_eclipse.in` holding a date to feed it:

```
# cd /root/cc
# mmbc solar_eclipse.bas
wrote solar_eclipse.c
# cc solar_eclipse.c
.....................................................
# ./solar_eclipse.bc < solar_eclipse.in
...
event duration          2.61100904 hours
Time taken :     2.168458  Seconds
```

The same program takes 12.5 seconds under MMBasic and 8.8 seconds under MicroPython on this hardware; every digit printed is identical in all three. Translating and compiling it together takes about twenty seconds — it is a hundred and forty kilobytes of C, so it is much the longest build in this manual.

This one was 1.95 seconds at v0.13, with rather less of MMBasic implemented; the digits of the answer have not moved.

6.5 What the translation looks like

The C is meant to be read. Variables keep their names, control flow keeps its shape, and the MMBasic semantics that C does not share — integer division, `Mod` on negative numbers, string handling, the 64-bit integer/double type rules — are handled by a small runtime inside `bcrun` rather than by rewriting your program into something unrecognisable.

```
# mmbc hello.bas
# head -20 hello.c
```

is worth doing once, to see what your program became.

6.6 Strings, arrays and functions

Strings are MMBasic strings: `Dim string s` gives the usual 255 characters, and `Dim string s length 40` the shorter form. Arrays index from `OPTION BASE` as they should, and `Bound()` reports their limits. `Sub` and `Function` work, including arrays passed by reference and modified in place.

Arrays and strings live in the PSRAM heap, so what limits them is the 8 MB rather than the size of a process. A framebuffer-sized array — `Dim Integer fb(4800)`, 38,408 bytes — is unremarkable now; before v0.6 it would not fit at all. Simple variables stay in the program's own memory, where they are quicker to reach, which is the same split MMBasic itself makes and for the same reason.

`LOCAL` arrays and strings get their own block for each call, released when the routine returns, so a recursive routine sees its own copy at every level. `STATIC` locals are unaffected — they are meant to outlive the call, so they stay where they were.

The one thing to remember is that a translated program is a *program*, not an interactive session: there is no editor, no `RUN`, no immediate mode. You edit the `.bas` file, translate, compile and run — which is also why a translated program can be put in `/usr/bin` and used like any other command.

6.7 Errors

`mmbc` reports what it could not translate, by line, and carries on:

```
# mmbc gpio.bas
2 line(s) could not be translated and were commented out:
  line 2: expected '='
  line 3: 'pin' is not an array
wrote gpio.c
```

The reasons are the translator's own — it reads `SetPin GP1, DOUT` as far as it can and then says what it wanted to see — so they name the symptom rather than the statement. The lines themselves appear in the C as comments, with the original text, which is the quickest way to see what was dropped:

```
    /*     SetPin GP1 , DOUT */
```

The translation still happens and everything else works. `--strict` stops on the first such line instead, and `--report` lists them again at the end along with any implied global variables.

Appendix C lists what is covered.

6.8 Graphics

A translated program draws on the HDMI display with MMBasic's own statements:

```
MODE 2
COLOUR RGB(YELLOW)
LINE 0, 0, 319, 239
CIRCLE 160, 120, 60, , , RGB(WHITE), RGB(BLUE)
PIXEL 10, 10, RGB(RED)
```

Two modes are available, chosen to match the VGA builds of MMBasic and the first two HDMI modes:

`MODE`	Resolution	Colours
`MODE 1`	640×480	monochrome
`MODE 2`	320×240	16, from MMBasic's RGB121 set

MODE	Resolution	Colours

A PIXEL outside the screen is **dropped**, which is what the interpreter does. Older releases here masked the coordinates instead, so PIXEL 1030, 100 appeared at x=6 — a program written against that will look different.

RGB() takes the usual named colours and RGB(r,g,b). COLOUR sets the current drawing colour, which every statement uses when no colour is given. LINE, CIRCLE and the rest take MMBasic's argument order, including the blank arguments — CIRCLE x, y, r, , , fill is written exactly as MMBasic writes it.

6.8.1 Shapes from arrays, and filling

```
DIM x(5), y(5)
POLYGON 6, x(), y(), RGB(WHITE), RGB(RED)   ' outline, then fill
POLYGON 0, x(), y()                          ' 0 = as many as the array holds
BEZIER cx(), cy(), , RGB(CYAN)               ' n control points
FILL 100, 80, RGB(GREEN)                     ' flood from a point
FILL 100, 80, RGB(GREEN), RGB(WHITE)         ' ...up to a boundary colour
```

POLYGON is always closed — the edge from the last vertex back to the first is drawn, and the fill assumes it. Coordinates may be integer or float arrays. **Concave outlines fill correctly**: the fill collects every edge crossing on each row and fills between alternate pairs, so an arrowhead's notch stays empty. The multi-polygon form, where the first argument is an array of vertex counts, is not translated.

BEZIER takes **integer** control-point arrays, which is MMBasic's own restriction, and at most sixteen of them.

FILL has MMBasic's two modes: given a boundary colour it fills everything reachable that is not that colour; without one it replaces the colour it finds under the starting point. Note that boundary mode fills *over* other colours — it stops only at the boundary. A circle interior takes about 6 ms and a whole 320×240 screen about 75 ms.

The drawing primitives live in headers as static functions, one header per primitive — mmb_gfx_circle.h, mmb_gfx_box.h, mmb_gfx_rbox.h, mmb_gfx_triangle.h, mmb_gfx_arc.h, mmb_gfx_text.h, mmb_gfx_map.h, over the shared batch helpers in mmb_gfx_pts.h — and the translator includes exactly the ones the program uses, so a program that never draws a circle does not carry the circle code. (cc discards a file-scope static nothing calls, but that rule counts names rather than reachability, so a recursive primitive survives inside any header that carries it — which is why the include, not the static, is the unit that matters.) And a whole shape crosses into the kernel in a single call, so a 640-point line is one syscall, not 640 — measured at 71 µs against 433 µs for the naive version.

The geometry is MMBasic's, copied rather than re-derived, down to the dotted appearance of a stretched ellipse. If a picture differs from MMBasic's, that is a bug worth reporting.

One caution. In MODE 1 the console and the graphics share a single framebuffer, so anything printed — including the cursor — appears on the picture. It is visible in a SAVE IMAGE of the whole screen as a difference in the top text line. Restricting the saved region avoids it; directing program output elsewhere is planned.

6.9 The sixteen colours: MAP

MODE 2 stores a colour *number* per pixel, 0 to 15, and MAP says what colour each number actually is. Change entry 8 and everything already drawn in colour 8 changes with it — without redrawing a single pixel. That is what makes fades, flashes and colour cycling free.

```
MAP(4) = RGB(255, 128, 0)      ' collect
MAP SET                        ' ... and apply, all at once
```

`MAP(n) = colour`	set entry `n`, 0 to 15. **Nothing changes yet**
`MAP SET`	apply the collected palette, during the frame blanking
`MAP RESET`	back to the mode's own sixteen
`MAP MAXIMITE`	the original Colour Maximite's sixteen
`MAP GRAYSCALE`	sixteen greys, 0 to 255 in steps of 17 (`GREYSCALE` too)

The two-step is the point, and it is MMBasic's own: applying a new palette one entry at a time shows the picture half recoloured, and `MAP SET` waits for the blanking so the change lands between frames.

`MAP(n)` is also a **function**, giving the colour entry `n` stands for by default — which is the colour a program must ask for to land on that entry:

```
FOR i = 0 TO 15
  LINE i * 20, 0, i * 20, 199, , MAP(i)     ' one line per entry
NEXT i
```

It reports the default and takes no notice of any remapping, exactly as MMBasic does; it is the inverse of the fixed rule that turns a colour into an entry number. That rule is bit extraction — one bit of red, two of green, one of blue — and it does **not** depend on the palette, so remapping never moves where new drawing lands.

`MAP` needs a 16-colour mode; in `MODE 1` it is an error.

6.9.1 COLOUR MAP — a whole array at once

`COLOUR MAP in%(), out%()` is the array form of the `MAP()` function: it turns a whole array of colour codes 0-15 into RGB888 colours in one statement, which is what an image or a tile table wants.

```
DIM INTEGER code(255), col(255)
COLOUR MAP code(), col()               ' the default palette
```

A third array replaces the palette for that call — it must be exactly sixteen entries, and each must be a valid 24-bit colour:

```
DIM INTEGER pal(15)
FOR i = 0 TO 15 : pal(i) = RGB(i * 17, i * 17, i * 17) : NEXT i
COLOUR MAP code(), col(), pal()        ' sixteen greys instead
```

The two arrays must be the same size, and `in` and `out` may be the same array — each element is read before it is written. The arrays are integer; MMBasic accepts float ones as well, and here that is an error rather than a silent conversion.

6.10 Text on the picture: TEXT and FONT

TEXT puts a string at a pixel position, justified how you ask, in any of nine fonts:

```
TEXT 160, 120, "Centred", "CM"
TEXT 319, 0, "top right", "RT", 3
TEXT 0, 100, "big and red", "LT", 5, 2, RGB(RED)
TEXT 0, 200, "over the picture", "LT", 1, 1, RGB(WHITE), -1
```

```
TEXT x, y, string$ [, alignment$] [, font] [, scale] [, colour] [, background]
```

Everything after the string may be left out, and a blank argument takes the default, as everywhere in MMBasic. background of -1 is transparent, which is how you write over a picture without a box of paper around the letters. With no colours given, COLOUR's are used.

alignment$ is up to three letters and any of them may be omitted:

	letters	meaning
horizontal	L C R	x is the left, the centre, the right
vertical	T M B	y is the top, the middle, the bottom
orientation	N V	normal, or one character per line downwards

The other three orientations MMBasic offers — I inverted, U and D rotated — are **not yet drawn**; they are accepted and come out normal.

The nine fonts are MMBasic's own, so a character looks the same here as it does there. They live in the PC3's flash, which is why there is no cost to having all of them:

font	cell	what it is
1	8×12	the console's own — the default
2	12×20	
3	16×24	
4	10×16	
5	24×32	
6	32×50	**digits only** — a clock face, a score
7	6×8	
8	4×6	the smallest
9	8×10	

scale is 1 to 15 and multiplies the cell, so font 1 at scale 2 is a 16×24 character built from the 8×12 one.

A character a font does not have prints as a blank cell. With font 6 that is everything except 0 to 9.

FONT sets the font PRINT itself uses, in a graphics mode:

```
FONT 3, 1
PRINT "large"
```

```
FONT [#]n [, scale]
```

This changes where the next line goes as well as how the letters look — a 24×32 font fills the screen in seven lines where font 1 takes nineteen — and scrolling follows it.

6.11 Your own font: `DefineFont`

A program can carry a font of its own and use it exactly like the nine built-in ones:

```
FONT 10
TEXT 8, 8, "SCORE 1000", "LT", 10

DefineFont 10
  03300808
  FFFFFFFF FFFFFFFF
  55AA55AA 55AA55AA
End DefineFont
```

The block may sit anywhere in the file — the bottom is the usual place, as it is in MMBasic, and the font is in force from the first line of the program whatever line it is written on.

Numbers 10 to 16. Fonts 1 to 9 are the built-in ones, shared with the shell and every other program, so they cannot be replaced; `DefineFont 9` is an error naming the range rather than a definition that quietly does not happen. Ports of PicoMite programs that define a low-numbered font need that one number changed.

The first word is the font describing itself and the rest are the glyphs, in MMBasic's format, so a `DefineFont` block from a PicoMite program can be pasted in unaltered:

the first word, byte by byte	
width in pixels	height in pixels
code of the first character	how many characters

Each group of eight hex digits is one 32-bit word, **least significant byte first** — so `03300808` is width 8, height 8, first character 0x30 (`0`), three characters. Each glyph follows as width × height bits, most significant bit leftmost, with no padding; width × height must be a multiple of 8. A font is checked when the program is translated, so a header that disagrees with the data it carries is a translation error rather than a screen full of rubbish.

The glyphs live in the program and cost nothing else — the firmware is handed their address, not a copy — and they go away with it. Another program asking for font 10 gets its own, or nothing.

6.12 The glyphs themselves: `MM.INFO(FONT ADDRESS)` and `PEEK`

`TEXT` draws on the PC3's own screen. If you have hung a display off the I/O header — an ILI9341 on SPI, say — the firmware knows nothing about it, and you are drawing every pixel yourself. You would rather not also carry your own copy of a font to do it.

You do not have to. Ask where the built-in ones are:

```
a = MM.INFO(FONT ADDRESS 3)
```

That is a machine address, and on this computer a machine address is something a program can simply read. There is no MMU and no memory protection: the fonts are `const`, so they live in flash where nothing can move them, and `PEEK` reaches them where they lie.

The first four bytes at that address are the font describing itself:

offset	
0	width in pixels
1	height in pixels
2	code of the first character it has
3	how many characters

so having the address you need nothing else — no table of sizes, no constants of your own to get wrong:

```
w     = PEEK(BYTE a)
h     = PEEK(BYTE a + 1)
first = PEEK(BYTE a + 2)
count = PEEK(BYTE a + 3)
```

The glyphs follow, each `width × height` bits, packed continuously with no padding between rows and no padding between characters, most significant bit first. The one for character c starts at

```
a + 4 + (c - first) * width * height / 8
```

and bit $y \times width + x$ of it is the pixel at (x, y). This is MMBasic's own layout, unchanged, because these are MMBasic's own nine fonts — the ones the console draws from.

Reading one back is the clearest way to see it. `fontaddr.bas` in `/root/MMBasic` does exactly this, and prints:

```
font  address     cell   first  count
 1    1000EF10    8x12     32    224
 2    1000F994   12x20     32     95
 3    100104BC   16x24     32     95
 5    10012814   24x32     32     95
 6    10014BB8   32x50     48     11
```

Font 6 saying `first 48, count 11` is the digits-only font describing itself: 48 is `"0"`, and eleven characters is `0` to `9` and one more. Nothing in the program was told that.

```
MM.INFO(FONT ADDRESS n)     n is 1 to 9; 0 if there is no such font
PEEK(BYTE addr)             one unsigned byte
PEEK(SHORT addr)            sixteen bits, signed
PEEK(WORD addr)             thirty-two bits, unsigned
PEEK(INTEGER addr)          sixty-four bits, signed
PEEK(FLOAT addr)            a double
```

Both spellings are MMBasic's, so a program written this way runs on a PicoMite unchanged.

These are sharp. A `PEEK` of a wrong address is not an error message — there is nothing on this machine to catch it. It reads whatever is there, or the program dies. Check the address you were given before you use it:

```
a = MM.INFO(FONT ADDRESS 3)
IF a = 0 THEN ERROR "no font 3"
```

The wider forms insist the address is a multiple of their width, as MMBasic does, and say `Address not divisible by 4` if it is not. That is a real check and not a formality: an unaligned load faults on this processor.

MMBasic's `PEEK(VAR ...)`, `PEEK(VARADDR ...)` and `PEEK(CFUNADDR ...)` are not translated — those ask about a *variable* rather than an address, which needs the symbol table. Neither is `POKE`, in any form. What else `MM.INFO` answers is listed under Asking the machine about itself.

There is a worked example of all of this driving a real panel under A worked example: a barometric station.

6.13 Animation: `FRAMEBUFFER`

Drawing straight to the screen means the monitor shows the picture half-finished — the erase, then each shape appearing in turn. MMBasic's answer is to build the frame off-screen and show it in one go, and it is here:

```
MODE 2
FRAMEBUFFER CREATE
FRAMEBUFFER WRITE F
DO
  CLS
  CIRCLE x, y, 20, , , 0, RGB(YELLOW)
  FRAMEBUFFER COPY F, N, B
LOOP WHILE INKEY$ = ""
```

`FRAMEBUFFER CREATE`	make the off-screen buffer, blank
`FRAMEBUFFER LAYER`	make the layer, blank
`FRAMEBUFFER WRITE N | F | L`	send drawing to the screen, the buffer, or the layer
`FRAMEBUFFER COPY s, d [, B]`	`s` and `d` each `N`, `F` or `L`; `B` starts at the top of the frame
`FRAMEBUFFER MERGE [colour]`	the layer over the buffer, onto the screen
`FRAMEBUFFER CLOSE [F | L]`	give one back
`FRAMEBUFFER WAIT`	wait for the top of the frame

Everything that draws follows `WRITE`, `CLS` included — so `CLS` while writing to `F` clears the buffer and leaves the picture on the screen alone.

`CLS [colour]`

fills whatever is being drawn on. With no colour it uses the background from `COLOUR`, so `COLOUR RGB(WHITE), RGB(BLUE)` then `CLS` gives a blue screen; `CLS RGB(GREEN)` fills with green without changing what `COLOUR` set. Either way the text cursor goes back to the top left, as MMBasic's does.

`CREATE` belongs **after** `MODE`. A mode change throws the buffer away, because what is in it is in the geometry of the mode being left and nothing converts it. That is MMBasic's rule too. You do not have to `CLOSE`: the buffer comes back automatically when the program ends.

Only one program can hold the buffer at a time — a second one is told so rather than quietly sharing it — and only the program holding it draws into it. Anything else that draws while your program is between frames, a shell prompt included, still goes to the screen.

What it costs. The buffer is in PSRAM, so drawing into it is about 2.4× the cost of drawing on the screen — a full-screen fill measured 1.49 ms direct against 3.63 ms buffered. The copy back is 38,400 bytes at about 53 MB/s, or **0.73 ms**. So a redraw-and-show frame costs somewhere near 4.4 ms against the 17 ms the display gives you, and `COPY ... ,B` holds the loop to the refresh rate: 59 frames a second, with room for roughly three times as much drawing before one is dropped.

6.13.1 The layer

`FRAMEBUFFER LAYER` makes a second off-screen buffer. It is the same size and shape as `F` and you draw into it the same way, with `FRAMEBUFFER WRITE L`; what makes it a *layer* is `MERGE`, which puts it **on top**:

```
FRAMEBUFFER MERGE [colour]
```

lays the layer over `F` and shows the result, treating `colour` as transparent — wherever the layer holds it, `F` shows through. Left out, it is 0.

Neither source is changed by a merge. That is the whole point of having one: the background goes into `F` once, the thing that moves goes into the layer, and a frame is one `CLS` of the layer, one shape, and one `MERGE` — no repainting of everything underneath:

```
MODE 2
FRAMEBUFFER CREATE
FRAMEBUFFER LAYER

FRAMEBUFFER WRITE F                    ' the background, drawn once
CLS RGB(BLUE)
BOX 20, 20, 120, 80, 2, RGB(WHITE), RGB(RED)

FRAMEBUFFER WRITE N
DO
  FRAMEBUFFER WRITE L
  CLS 0                                ' 0 is the transparent index
  CIRCLE x, 120, 30, 2, 1, RGB(YELLOW), RGB(YELLOW)
  FRAMEBUFFER WRITE N
  FRAMEBUFFER MERGE 0
  x = x + 4
LOOP WHILE INKEY$ = ""
```

`COPY` takes `L` at either end, so `COPY F, L` puts the background into the layer to draw over, and `COPY L, N` shows the layer on its own. `CLOSE L` gives it back; so does the end of the program.

The transparent colour means what it means in the mode you are in. In `MODE 2` a pixel is four bits, so it is a colour index from 0 to 15 and the merge keys on it per pixel. In `MODE 1` a pixel is one bit: transparent 0 means the layer's set pixels paint and its clear ones let `F` through, and transparent 1 is the other way round. Same command, same program, and `MODE 1` is the faster of the two.

What it costs. A merge waits for the top of the frame first, always — it writes into the buffer being scanned out, and starting part way down tears the picture once per merge. In `MODE 2` the loop above runs at **16.62 ms a frame, 60 frames a second**, of which the drawing is 2.57 ms; the rest is the composite and the wait it is hidden behind. It is quantised by that wait, so a little more drawing costs nothing at all and too much costs a whole frame.

`MERGE` needs both buffers and says so — "Layer not created" if there is no layer, "Frame buffer not created" if there is no `F` — rather than doing half of it.

This is MMBasic's TFT model, the one its ILI9341 builds use, so a program written for a PicoMite with a panel on it runs here unchanged. Its VGA and HDMI builds do the same compositing inside the scanline builder instead, continuously and without a command; that needs every buffer in the RAM the

display DMAs from, which on this machine is 40K taken off every program forever, whether it uses a layer or not. The reasoning is written out in PC3-LAYER-MERGE.md in the source tree.

INKEY$ returns the key that has been pressed, or "" if none has, without waiting — which is how the loop above ends. It leaves the terminal as it found it, so INPUT still works afterwards and a program stopped with Ctrl-C does not take the shell's echo with it.

A cursor or function key reaches a terminal as an escape sequence of several bytes, and INKEY$ reassembles it and gives back the single code MMBasic gives, so the PicoMite idiom works unchanged:

```
DO
  k$ = INKEY$
  IF k$ = CHR$(&H80) THEN PRINT "up"
  IF k$ = CHR$(&H91) THEN PRINT "F1"
LOOP UNTIL k$ = "q"
```

The codes are MMBasic's: &H80–&H83 for up, down, left and right, &H84 insert, &H86 home, &H87 end, &H88/&H89 page up and down, &H7F delete, and &H91–&H9C for F1 to F12. A sequence it does not recognise comes back byte by byte behind its escape, so nothing is lost — and a bare ESC is still CHR$(27).

6.14 Running another program: SYSTEM, SAVE IMAGE, LOAD IMAGE, LOAD JPG, LOAD PNG

SYSTEM runs a program and waits for it:

```
SYSTEM "sum", "a.bmp", "b.bmp"
```

Each argument is passed separately, so no shell quoting is involved and a filename with a space in it needs no special care.

SAVE IMAGE and LOAD IMAGE are built on it — they run the saveimage and loadimage programs described in the applications list, which means a BASIC program that never touches an image pays nothing for them:

```
SAVE IMAGE "shot.bmp"                       ' the whole screen
SAVE IMAGE "part.bmp", 160, 120, 320, 240   ' x, y, w, h
LOAD IMAGE "shot.bmp"                       ' at 0,0
LOAD IMAGE "shot.bmp", 160, 120             ' at x,y
```

LOAD IMAGE takes MMBasic's full syntax including the dither mode and the source rectangle:

```
LOAD IMAGE fname$ [,x] [,y] [,mode] [,ximage] [,yimage] [,wimage] [,himage]
```

The decoder is MMBasic's, so it reads 1, 4, 8, 16, 24 and 32-bit BMPs, BI_BITFIELDS, and RLE4/RLE8 compression. Dithering is *optional*, as in MMBasic — omit the mode and the image is mapped without it.

LOAD BMP is accepted as a synonym for LOAD IMAGE, as in the reference.

6.14.1 LOAD JPG

```
LOAD JPG fname$ [,x] [,y] [,mode] [,ximage] [,yimage] [,scale]
```

MMBasic's own picojpeg, as the `loadjpg` program. `scale` is 1, 2, 4 or 8 and bins that many source pixels into one on screen, so a photograph larger than the display can be fitted to it. `ximage`/`yimage` take a region out of the middle of a larger picture.

Binning costs nothing extra: every MCU still has to be decoded, because a JPEG stream cannot be seeked into — only the drawing is reduced. A 320×240 photograph draws in about 0.45 s either way.

`mode` is the dither argument. It is *parsed and ignored*, for the reason given under `LOAD IMAGE`: error diffusion wants rows of signed error per channel, and this program runs while your BASIC program is still resident in memory.

6.14.2 LOAD PNG

```
LOAD PNG fname$ [,x] [,y] [,transparent] [,cutoff]
```

MMBasic's own upng, as the `loadpng` program.

`transparent` decides what happens to see-through pixels, and its default will surprise you the first time. As in the reference it defaults to **0**, which is the palette's black — so a PNG with a transparent surround lands on screen inside a black box. That is what a PicoMite does, and it is deliberate: the same default is what makes `SPRITE LOADPNG` work (see below). To let the screen show through instead, pass **-1**:

```
CLS RGB(255,255,255)
LOAD PNG "apple.png", 0, 0, -1     ' the white shows through
LOAD PNG "apple.png"               ' a black box around it
```

`cutoff` is the alpha value above which a pixel counts as opaque, 1 to 254, default 20.

Note that an *opaque* black pixel is drawn black whichever you choose — nothing can tell "background" from "black" once the alpha channel says opaque. If a picture has an unwanted dark surround even with `-1`, the file has an opaque background rather than an alpha one.

`LOAD PNG` needs the PSRAM arena and says so if a kernel cannot provide one. PNG cannot be decoded a piece at a time — its filters refer to the row above and its compressed stream is one window over the whole image — so the entire picture is inflated before any of it can be drawn: about 300 KB for a full screen, twice the whole process pool. The reference has the same constraint and offers `LOAD PNG` only on RP2350 for it.

6.15 Music: PLAY MP3, PLAY WAV, PLAY FLAC, PLAY SOUND, PLAY TONE, PLAY MODFILE

```
PLAY VOLUME 70
PLAY MP3 "/root/mp3/whiter.mp3"
' the program carries straight on, and the music plays
FOR i% = 1 TO 20
  PRINT "still working"; i%
  PAUSE 500
NEXT i%
PLAY STOP
```

`PLAY MP3` does **not** wait. The decoder is a separate program feeding a 1.5-second buffer in the kernel, and the sound hardware empties that buffer by DMA, so playback needs nothing from your program once it has started. MMBasic has to refill its audio from the interpreter's idle loop; here there is no idle

loop to do it in, and no cost when there is nothing to play. A translated benchmark measured **89% of its full speed** with a track playing.

`PLAY WAV` and `PLAY FLAC` are the same statement with a different decoder behind it, and behave the same way in every respect — they do not wait, `PLAY STOP` stops them, and `PLAY VOLUME` sets their level. WAV covers 8, 16, 24 and 32-bit PCM, IEEE float, A-law and mu-law, all converted to the 16 bits the hardware takes. FLAC is read at its own rate and bit depth; a 44.1 kHz 16-bit stereo file plays with no underruns while the machine is otherwise idle.

One thing to know about FLAC in particular: its decoder is sized from the file, at roughly `maxBlockSize x 4 x channels`, so a file written with 4096-sample blocks wants about 32K and one written with 16384 wants about 132K — out of a pool of 336K shared with everything else running. A file that will not open says so, and says how much it wanted, rather than failing silently.

`PLAY VOLUME n` takes 0 to 100 and is remembered, so every later `PLAY MP3`, `PLAY WAV` or `PLAY FLAC` uses it until it is changed again. Out-of-range values are clamped rather than refused. The default is 80. The scale is logarithmic, so 50 is a comfortable half rather than a whisper.

`PLAY STOP` stops whatever is playing. It asks the kernel which process holds the sound output rather than remembering what it started, so it also stops a player left running by a program that was stopped with Ctrl-C — which is the situation it is most often wanted for. Stopping when nothing is playing is not an error, as in MMBasic.

There is one sound output, so there is one player:

```
Error: sound output in use
```

is what `PLAY MP3` gives if something is already playing. Use `PLAY STOP` first. (This is MMBasic's "Sound output in use for ..." under a shorter name.) The rule is enforced by the kernel rather than by convention: two decoders writing into the same buffer interleave their samples, and it sounds like it.

Files play at their own sample rate — 8 kHz to 48 kHz, mono or stereo, and a mono file costs nothing extra because the driver duplicates the channels itself. Copy MP3s onto the FAT partition from a PC and bring them over with `fat get`, as with any other file.

`PLAY` and BBC BASIC's `SOUND` share one piece of hardware and are mutually exclusive, exactly as they are in MMBasic. Starting a track silences the synthesiser; stopping it hands the hardware back.

6.15.1 The synthesiser: `PLAY SOUND` and `PLAY TONE`

```
PLAY SOUND 1, B, S, 440, 15     ' voice 1, both sides, sine, 440 Hz
PLAY SOUND 2, B, Q, 220, 10     ' a square underneath it
PAUSE 1000
PLAY SOUND 1, B, O              ' voice 1 off
PLAY TONE 440, 554, 600         ' left/right pair for 600 ms
```

MMBasic's four-voice synthesiser, rendered by the kernel itself in the audio interrupt that drives the DAC — the same place MMBasic renders it. Four voices, each with an independent left and right side; channels are `L`, `R`, `B` or `M`, and the types are `Sine`, `Q` (square), `Triangle`, `W` (sawtooth), `Periodic noise`, `N` (white noise) and `Off`. Frequency is 1 Hz to 20 kHz, volume 0 to 25. `PLAY TONE` takes left and right frequencies and an optional duration in milliseconds, rounded down to whole cycles as MMBasic rounds it; no duration means until further notice. The square and sawtooth are band-limited (polyBLEP), so they hold a pure pitch at the top of the range instead of shimmering.

Channel and type may be written as a bare letter, a quoted letter or a string worked out at run time, as MMBasic writes them: `PLAY SOUND 1, B, S, 440`, `PLAY SOUND 1, "B", "S", 440` and `PLAY SOUND 1, c$, t$, 440` are the same statement.

A change of note is heard at once. Until v0.15 the synthesiser was a separate program rendering ahead into a queue, which put a fifth of a second between the statement and the sound — audible as lag in a game, and the reason a program changing pitch every 20 ms could not work at all. It now runs where MMBasic runs it, in the interrupt that feeds the DAC, so a `PLAY SOUND` is in the next buffer the hardware asks for: under two milliseconds. The voices go quiet by themselves after five seconds with every voice off, or on `PLAY STOP`, and a program that ends — including one stopped with `Ctrl-C` — takes its sound with it.

6.15.2 Modules: `PLAY MODFILE` and `PLAY MODSAMPLE`

```
PLAY MODFILE "/root/ptetris.mod"
PAUSE 5000
PLAY MODSAMPLE 3, 4             ' fire sample 3 on channel 4, over the music
PLAY STOP
```

`PLAY MODFILE` plays ProTracker modules at 22 kHz through the hxcmod tracker, the whole file staged in PSRAM. Like MP3 it runs as its own program and does not wait. `PLAY MODSAMPLE n, ch [, vol]` asks the *running* player to mix one of the module's own samples over the music - MMBasic's game sound-effect trick - with nothing loaded and nothing interrupted. With a completion interrupt the module plays once; otherwise it loops until `PLAY STOP`.

6.16 Pins: `SETPIN` and `PIN`

```
SETPIN 2, DOUT
SETPIN 3, DIN
PIN(2) = 1
IF PIN(3) = 1 THEN PRINT "high"

SETPIN 41, AIN                  ' analogue, in volts
PRINT PIN(41)                   '   1.677936508
SETPIN 41, ARAW                 ' analogue, raw
PRINT PIN(41)                   '   2090
```

All **forty-eight** GPIOs of the RP2350B are available — the wider part is why the PC3 can put the real-time clock's alarm on GP32, which a 28-pin RP2040 could not reach.

A pin is a register access, not a system call — about ten nanoseconds against the 1.5 microseconds a crossing into the kernel costs. That is what makes bit-banging a protocol from BASIC possible at all, and it is why `ONEWIRE` and the rest are written in your own program's time rather than as kernel drivers. The kernel still owns the pin: `SETPIN` claims it, claiming one another program holds is refused, and the claim is released — with the pin **reset to an input** — when your program ends, however it ends. A program that dies driving a relay does not leave it driven.

`n` is the **GPIO number**, not MMBasic's connector-pin numbering. The GPIO number is what the schematic, the kernel, and every other tool on this machine use, and inventing a second numbering for one statement would cause more confusion than the incompatibility does.

`SETPIN` takes:

mode	what the pin becomes
`DIN`	a digital input
`DOUT`	a digital output, driven by `PIN(n) = v`
`AIN`	an analogue input read as **volts**
`ARAW`	an analogue input read as the **raw 0–4095 count**
`INTH`	a digital input that calls a SUB on a **low-to-high** edge
`INTL`	... on a **high-to-low** edge
`INTB`	... on **either** edge
`PWM`	a PWM output, driven by the `PWM` statement
`OFF`	not configured

MMBasic's remaining modes — frequency and counting — are not translated, and are reported by name.

Input modes take MMBasic's optional pull:

```
SETPIN 2, DIN, PULLUP
SETPIN 3, DIN, PULLDOWN
SETPIN 35, INTL, Pressed, PULLUP      ' after the handler
```

Absent means neither, which is MMBasic's default — a floating input then reads whatever the wire is doing, and with nothing connected that is noise. A switch to ground wants `PULLUP`.

Hysteresis is always on for inputs. It is not an option in MMBasic either: every digital input it configures gets the Schmitt trigger, and so does every one here. On a board with header pins and flying leads a slow or noisy edge is the normal case.

`AIN` and `ARAW` need an ADC pin: on the RP2350B channel n is GP40+n, so **GP40–GP46** on the I/O header. Anything else gives `Pin cannot do that`.

The two analogue modes differ only in what `PIN()` gives back. `ARAW` is a single conversion, which is what you want when you are going to do your own filtering or you care about speed. `AIN` is MMBasic's filter, step for step: ten readings, sorted, the top two and bottom two thrown away, and the remaining six averaged and scaled by 3.3 V. The point of it is the *discard* — one wild sample is dropped rather than smeared across the answer, which plain averaging would do. (MMBasic's `OPTION VCC` is not supported, so the scale is always 3.3 V.)

`PIN()` returns a **float in every mode**, where MMBasic returns an integer for digital pins and `ARAW`. Nothing observable changes — `PRINT PIN(2)` still prints `1`, and comparisons and array indices behave the same — because a double holds 0, 1 and every 12-bit count exactly. The reason is that translated C has to know the type when it is generated, and nothing at that point knows what mode a pin will be in.

Pins are now owned. `SETPIN` claims the pin from the kernel, and claiming one that another program holds gives `Pin cannot do that` rather than quietly fighting over it. The claim is released — and the pin **reset to an input** — when your program ends, however it ends, including being killed or crashing. So a program that dies driving a relay does not leave it driven.

Only the I/O header can be claimed: **GP0–GP7, GP26 and GP34–GP46**, plus **GP32**. The rest belong to the board (display, SD card, PSRAM, sound, console, the DS3231's I2C), and asking for one gives `Pin cannot do that` instead of taking a pin the audio hardware is driving. Appendix A lists what the board uses. A pin number outside 0–47 gives `Invalid pin`.

GP32 is not on the header — it is the **DS3231's alarm output**, and an alarm nothing can read is not an alarm. It is open-drain and active low, so with a pull-up it reads 1 until the alarm asserts:

```
SETPIN 32, DIN, PULLUP
PRINT PIN(32)                    ' 1 = not asserted

SETPIN 32, INTL, WakeUp, PULLUP   ' fires when it does assert
```

6.16.1 Arming the alarm

MMBasic has no alarm command — a program writes the chip's registers, and so does this:

```
SUB Alarm
  hits = hits + 1
  RTC SETREG 15, 0            ' clear the flag, or it never fires again
END SUB

RTC SETREG 7, 128             ' alarm 1 masks: match nothing = every second
RTC SETREG 8, 128
RTC SETREG 9, 128
RTC SETREG 10, 128
RTC SETREG 15, 0              ' clear any stale flag
RTC SETREG 14, 5              ' INTCN | A1IE - drive INT, enable alarm 1

SETPIN 32, INTL, Alarm, PULLUP
```

`RTC GETREG reg, var` and `RTC SETREG reg, value` reach any of the DS3231's registers, which is MMBasic's own interface (it has `GETREG` and `SETREG` and no alarm command). Register 7–10 are alarm 1, 0x0E is control, 0x0F is status. The chip's data sheet is the reference.

Your handler must clear the alarm flag. The INT line stays low until it is, so an alarm that does not clear it fires once and then looks broken.

Two things this cannot do. It will not stop the oscillator — a write to the control register comes back with `EOSC` masked out, because on a battery-backed part a stopped clock outlives the power cycle and the machine boots not knowing the time. And it is not `/dev/i2c`: that driver refuses address 0x68 outright, so this goes through the kernel's own path to the chip, sharing the retry and recovery the system clock already relies on.

On a **Pico Computer 2** GP32 is the SD card's MISO instead, so the same kernel refuses to hand it out there — the board tells itself apart at boot. This is the one pin whose availability depends on which machine you are running on.

`PIN(n) = v` on a pin that is not a `DOUT` gives `Pin is not an output`, and reading a pin that has never been `SETPIN`ed gives `Pin is not an input` — both as MMBasic does.

Pin work no longer costs a system call. `SETPIN` makes one, once, to claim; after that `PIN(n)` and `PIN(n) = v` are single register accesses, about ten nanoseconds against the 1.5 µs an ioctl costs. That is what makes bit-banging a protocol in BASIC possible at all — and it is not a theoretical claim: `ONEWIRE` is bit-banged that way, in slots of 60 µs with a 10 µs sample point inside them, which an ioctl per edge could not have fitted twice over.

6.17 Pin interrupts

```
SUB Pressed
  PRINT "GP35 went ";PIN(35)
END SUB

SETPIN 35, INTB, Pressed         ' either edge
DO
  ' ... ordinary work; the handler runs between statements
LOOP
```

The handler is an ordinary `SUB` taking no parameters, and it ends with `END SUB`. **There is no `IRETURN` to write** — that is MMBasic's behaviour too, not a simplification: for a SUB target MMBasic builds an `IRETURN` of its own and uses it as the return address of the `GOSUB` it fakes, so `END SUB` performs the interrupt return. Written `IRETURN` only ever existed for targets given as a label or a line number, and those are not translated: compiled code cannot jump into the middle of a function, so a label or line number is refused with a clear message rather than half-working.

These are not hardware interrupts, and they are not in MMBasic either. The whole facility is a poll. MMBasic's interpreter checks after every statement, and pin "interrupts" there are a comparison of the pin's level against its level at the previous check — there is no GPIO IRQ anywhere in it. This does the same thing at the same place, so the behaviour you get is the behaviour a PicoMite gives:

- **Latency is one statement**, and a statement is atomic. Nothing ever runs half a statement of your program.
- **A pulse shorter than a statement is missed.** That is MMBasic's documented limitation, replicated rather than "fixed".
- **Interrupts never nest.** A handler's own statements are polled, but the poll does nothing while a handler is running.
- **One handler runs per statement boundary.** If two pins change at once, the second fires at the next statement.
- `MM.ERRNO` and `MM.ERRMSG$` are **saved, cleared and restored** around a handler, so a handler starts clean and cannot leave an error behind for the interrupted code to trip over.

Up to ten pins may have interrupts, which is MMBasic's own limit. The level a handler reads with `PIN(n)` is the level *now*, not the level that triggered it — with a mechanical switch the two often differ, because contacts bounce faster than the handler starts. MMBasic reads it the same way.

A program that arms no interrupt pays nothing at all: the per-statement poll is emitted only for programs that use the feature, exactly as the `ON ERROR` checks are.

6.18 `PWM`

```
SETPIN 0, PWM                ' GP0 is slice 0, channel A
PWM 0, 1000, 25              ' slice 0: 1 kHz, 25% on channel A
PWM 0, 1000, 25, 75          ' ...and 75% on channel B
PWM 0, 1000, -25             ' negative duty = inverted output
PWM 0, OFF
```

`SETPIN pin, PWM` attaches a pin to its PWM output; the `PWM` statement then drives the **slice**, not the pin. One slice has two channels and drives two pins, which is why the two commands are separate — the same split MMBasic uses.

Which slice a pin belongs to is arithmetic: pins below GP32 use slice `(pin ÷ 2) MOD 8`, and GP32 upward use `8 + ((pin ÷ 2) MOD 4)`. Even pins are channel A, odd pins channel B. **Twelve slices cover forty-eight pins, so pins alias**: GP34 and GP42 are the same slice and channel, and setting one sets the other. `SETPIN pin, PWM` claims the slice as well as the pin, so a second program asking for a pin that lands on a slice you hold is told `Pin cannot do that` rather than quietly changing your frequency.

Frequency is in Hz and duty in percent, and the range is wide: 100 kHz and 50 Hz both work. Below about 5.7 kHz the 16-bit counter cannot hold a whole period at 375 MHz, so the clock divider takes up the slack — that happens automatically, and the only sign of it is that very low frequencies have coarser duty resolution. Asking for something the divider cannot reach either gives `Invalid frequency`.

A **negative duty** inverts that channel's output: `-25` is 25% inverted, so the pin reads high 75% of the time. It is the channel's polarity bit, not a recomputed duty.

`PIN()` will not read a PWM pin — it is an output, and MMBasic refuses it too.

Giving only one duty leaves the other channel alone. That is not a detail: two outputs share a slice, so `PWM 0, 1000, 25` must not stop whatever is running on channel B.

6.19 SERVO

```
SETPIN 0, PWM
SERVO 0, 50                 ' centre
SERVO 0, 0, 100             ' two servos on one slice, opposite ends
SERVO 0, OFF
```

A servo is PWM at a fixed 50 Hz frame with the position expressed as a pulse width, so `SERVO` is `PWM` with one line of arithmetic in front of it — MMBasic's:

position	pulse	
0	1.0 ms	one end
50	1.5 ms	centre
100	2.0 ms	the other end
-20	0.8 ms	over-travel
120	2.2 ms	over-travel

The range is -20 to 120, which is the over-travel most servos will accept; outside it you get `Invalid servo position`. Everything else is the `PWM` statement, including the rule that omitting the second position leaves that channel alone — which is what lets two servos share a slice.

Scope-verified on GP0 at all five positions.

6.20 SETTICK — periodic timers

```
SUB Every100
  INC count
END SUB

SETTICK 100, Every100          ' every 100 ms
SETTICK 100, Every100, 2       ' timer 2 of 4
SETTICK PAUSE, 2               ' freeze it where it stands
```

```
SETTICK RESUME, 2
SETTICK 0, 0, 2                 ' off
```

Four timers, numbered 1 to 4, period in milliseconds — MMBasic's `NBRSETTICKS`. The number is optional and defaults to 1. The handler is a parameterless SUB, as for pin interrupts, and everything said above about them applies here too: it runs between statements, it does not nest, and one interrupt is dispatched per statement boundary. A pin edge and a tick falling due at the same moment dispatch the pin first, which is MMBasic's scan order.

Missed ticks are dropped, not queued. If a handler takes longer than its own period, the timer catches up by whole periods and fires once — so a slow handler cannot spiral into a backlog it never clears. The phase is kept, so the cadence does not drift.

Measured on the board: twenty ticks of 100 ms took **1999.98 ms**, and a 10 ms timer running beside a 50 ms one fired exactly 100 times in the second the slow one took to reach twenty.

6.21 `ON KEY` — the console

```
SUB AnyKey
  k$ = INKEY$                   ' the key is still there for you
END SUB

SUB QuitKey                     ' fires on "q" only, and eats it
  finished = 1
END SUB

ON KEY AnyKey                   ' any key
ON KEY 113, QuitKey             ' just this one
ON KEY 113, 0                   ' that one off
ON KEY 0                        ' the any-key one off
```

Two forms, and **what happens to the key is the difference between them** — this is MMBasic's design, not an accident of ours:

- `ON KEY handler` fires while a key is *waiting*, and the key stays waiting. Your handler reads it with `INKEY$`. If it doesn't, the handler fires again at the next statement, because the key is still there.
- `ON KEY code, handler` fires only on that character code, and **consumes** it. It never reaches `INKEY$`, so a key reserved for a hot-key handler cannot also turn up in the program's ordinary input.

The specific form is checked first, so with both armed the reserved key goes to its own handler and everything else goes to the general one.

Two things to know before relying on it:

The console is checked at most every 5 ms, not after every statement. Looking is a system call, and a poll site runs after every statement; checking each time would cost far more than most statements do. Five milliseconds is the kernel's own tick and far below what anyone can type or perceive, but it is a divergence from MMBasic, which looks every time.

A program reading keys holds the terminal, and that has visible consequences worth knowing.

The console has a line editor in front of it — the thing that gives you history and cursor keys at the shell prompt. It takes every keystroke while the terminal is in cooked mode with echo, and hands over a

finished line only when you press Enter. A program that wants *keystrokes* has to stand it down, so the first `INKEY$` or armed `ON KEY` puts the terminal into raw mode and **keeps it there** for the rest of the program.

While it is held:

- **keys are not echoed** — nothing appears on screen unless your program prints it. This is MMBasic's behaviour for `INKEY$` too.
- **there is no line editing** — no backspace, no history, no cursor keys. Each keystroke goes straight to the program.
- **`INPUT` still works normally.** It gives the terminal back for the duration of the question and takes it again afterwards, so a program can mix `INKEY$` with `INPUT` and the typed answer is edited and echoed as usual.
- the terminal is **restored when the program ends**, however it ends.

This is what BBC BASIC and the editors on this machine already do.

6.22 `KEYDOWN` — which keys are *held*

`INKEY$` tells you what was **typed**. `KEYDOWN` tells you what is **down right now**, and for a game that is a different question: a character stream has no way to say "up and fire together".

```
DO
  IF KEYDOWN(0) THEN                    ' anything at all held?
    FOR i = 1 TO KEYDOWN(0)
      k = KEYDOWN(i)                    ' 1 is the most recent
      IF k = 128 THEN y = y - 1         ' up
      IF k = 129 THEN y = y + 1         ' down
      IF k = 130 THEN x = x - 1         ' left
      IF k = 131 THEN x = x + 1         ' right
      IF k = 32  THEN Fire               ' ... at the same time
    NEXT i
  ENDIF
LOOP
```

argument	answer
`KEYDOWN(0)`	how many keys are held, 0 to 6
`KEYDOWN(1)` ... `KEYDOWN(6)`	the code of the *n*th, 1 being the most recent
`KEYDOWN(7)`	modifier bitmap: 1 left Alt, 2 left Ctrl, 4 left GUI, 8 left Shift, and 16/32/64/128 for the right-hand four
`KEYDOWN(8)`	locks: 1 caps, 2 num, 4 scroll

Six is the hardware's limit, not ours. A USB keyboard's boot report carries six concurrent key codes, which is exactly the six MMBasic exposes, so this is the report itself rather than anything reconstructed. Most keyboards also refuse certain three-key combinations for reasons of their own wiring; that is the keyboard, and no software can see past it.

`KEYDOWN` empties the type-ahead buffer, exactly as MMBasic does. That is not tidiness: a held key still sends characters, and a game that read only `KEYDOWN` would otherwise pile up minutes of them behind the terminal, to be delivered to the shell the moment it exited. The consequence is that `KEYDOWN`

and INKEY$ are **alternatives, not layers** — whichever asks first gets the keystroke. Pick one per program.

Like INKEY$, the first KEYDOWN puts the terminal in raw mode and holds it, with everything the section above says about that.

6.23 Asking the machine about itself: MM.INFO

```
PRINT MM.DEVICE$                ' Fuzix
PRINT MM.INFO(PLATFORM)         ' PC3   (or PC2)
PRINT MM.INFO(VERSION)          ' same number as MM.VER
```

MM.DEVICE$ is "Fuzix" — the firmware, and nothing else. MMBasic answers with its build (PicoMiteVGA, PicoMiteHDMIUSB), and a program's whole use of it is to pick a path: IF INSTR(MM.DEVICE$, "PicoMite"), IF INSTR(MM.DEVICE$, "VGA"). Every one of those questions has the answer "no" here, and one word says so cleanly enough to test with a plain comparison. **Which machine** it is has moved to MM.INFO(PLATFORM), which is where MMBasic keeps it too — DEVICE is the firmware, PLATFORM is the board.

MM.INFO(...) and MM.INFO$(...) are the same function, as in MMBasic: the sub-keyword decides the type, not the $.

sub-keyword	answer
DEVICE	"Fuzix"
PLATFORM	"PC3" or "PC2"
VERSION	the release, as MM.VER
PATH	the directory the running program was started from, ending in /
CURRENT	the program's own file name
DRIVE	always "A:" — see below
EXISTS FILE f$	1 a file, **-1 a directory**, 0 nothing there
EXISTS DIR d$	1 if it is a directory
FILESIZE f$	bytes; -1 nothing there, -2 a directory
OPTION BASE	0 or 1
PINNO "GP8"	the pin a name stands for
FONTHEIGHT, FONTWIDTH	the current font's cell **times the current scale**
HPOS, VPOS	the graphics text cursor
ERRNO, ERRMSG	as MM.ERRNO and MM.ERRMSG$
FLAGS	all sixty-four scratch bits
FONT ADDRESS n, FLASH ADDRESS n	as before

MM.FONTHEIGHT, MM.FONTWIDTH, MM.HPOS and MM.VPOS are the flat spellings of four of those, and MMBasic has both — a program may write either.

DRIVE is always "A:", and that is not a pretence. This machine has one filesystem and no drive letters. Programs that ask save the drive, do a file operation and put it back; a constant answer makes that round trip *correct* rather than merely harmless.

The -1 from EXISTS FILE is MMBasic's own, and it earns its keep: it is how a program tells "that name is a directory" from "there is nothing there" without a second call.

FONTHEIGHT and FONTWIDTH follow FONT, and include the scale — so INC y, MM.INFO(FONTHEIGHT) + 1 moves down one line whatever font is selected, which is what the shape is for.

Everything else MMBasic's MM.INFO reports is about hardware, a flash program store or a network that is not here.

6.24 Pin names: GP8

Wherever a pin is expected you may write its **name** instead of its number:

```
SETPIN GP4, DIN, PULLUP
IF PIN(GP4) = 0 THEN PRINT "pressed"
x = PORT(GP0, 8)                    ' GP0..GP7 as one number
n = MM.INFO(PINNO "GP" + STR$(i)) ' ... or build the name at run time
```

On this machine a pin **is** its GPIO number, so GP8 is simply 8 — MMBasic needs a lookup table here because its pin numbers are connector pins. A variable you have declared with that name still wins.

This closes a trap rather than only adding convenience. Before it, GP8 in a program without OPTION EXPLICIT became an implied variable worth zero, so PIN(GP8) quietly read **GP0** — a wrong pin, with nothing said. A silent divergence outranks a missing feature, and this one is now neither.

6.25 Timers and the deliberate divergence

The 100 ms figure above is the one **deliberate divergence** from MMBasic, and it is in your favour. MMBasic counts milliseconds in an interrupt and fires when the count is *greater than* the period, so a SETTICK 100 there actually runs at 101 ms and a 16 ms game timer at 17 ms — about 6% slow. This keeps a microsecond deadline instead of a millisecond counter and fires at the period you asked for. Nothing else about the behaviour differs; if you are porting a program whose timing was tuned against a PicoMite, it will run very slightly faster here.

PAUSE gives the CPU up rather than spinning, so a waiting program does not hold the machine against everything else on it. The fraction it cannot sleep through is bounded at 99 ms however long the pause. **And PAUSE services interrupts while it waits**, as MMBasic's does: that matters more than it sounds, because a program that arms a SETTICK usually has a main loop of little but PAUSE, and a handler in such a loop would otherwise never run at all.

Every read and write is a call into the kernel — comparable to drawing one pixel, which is over a microsecond. That is ample for a switch or an LED and nowhere near enough to bit-bang a protocol; write that in C, or use the hardware that already exists for it.

6.26 Practical notes

- One .bas file per program, because cc compiles one file per program. Subs and Functions all live in that file.
- The C file is a normal file — you can keep it, edit it, or hand it to cc on another day.
- mmbc and cc each need a little room. On a machine with other things running, a large program is happier if you are not also holding a big file open in an editor.
- Programs built this way are the same objects cc produces from hand written C, so bcdump disassembles them and BCRUN_BYTECODE=1 forces interpretation for comparison.

6.27 A worked example: a barometric station

Everything in this chapter, on one screen. The program is `qnh.bas` in `/root/MMBasic`; what follows is how it is built rather than a listing of it.

It reads a BMP180 barometer, a potentiometer and the clock, and puts the results on an ILI9341 panel that the firmware has never heard of — including the text, drawn from the built-in fonts:

```
GP2  SCLK    GP3  MOSI    GP4  MISO     the panel
GP5  DC      GP6  RESET   GP7  CS       its control lines
GP0  LED                               backlight, PWM slice 0
GP38 SDA     GP39 SCL                  I2C2, the BMP180 at &H77
GP41                                   the potentiometer wiper
```

Build and run it the usual way:

```
# mmbc qnh.bas
# cc qnh.c
# ./qnh.bc
```

6.27.1 What it is for

A barometer reads the pressure where it is standing. Weather reports quote pressure **reduced to sea level**, because otherwise a reading would say more about your altitude than about the weather — the difference is about 12 hPa per hundred metres, which is the gap between a settled day and a gale.

The reduction needs to know how high you are, and that is what the potentiometer sets. The formula is the one in the BMP180's own datasheet, inverted:

```
qnh = pHPa / (1 - alt / 44330.0) ^ 5.255
```

The result is QNH: what an altimeter should be set to so it reads height above sea level.

6.27.2 Reading the sensor

The BMP180 arithmetic is Bosch's, out of the datasheet, and the program uses MMBasic's own BMP180 listing unchanged apart from the bus — the part is on the I/O header rather than the QWIIC socket, so `SETPIN` names the pins and `I2C2` opens the second controller:

```
SETPIN 38, 39, I2C2
I2C2 OPEN 400, 1000
I2C2 WRITE &H77, 1, 1, &HAA        ' the calibration block
I2C2 READ  &H77, 0, 22, i2cin$
ac1 = STR2BIN(int16, MID$(i2cin$, 1, 2), big)
```

Twenty-two bytes of calibration come back as a string and `STR2BIN` picks signed and unsigned, big-endian words out of it.

6.27.3 Reading the potentiometer

One line of setup, because `AIN` is MMBasic's:

```
SETPIN 41, AIN
alt = INT(PIN(41) / 3.3 * 1000 + 0.5)
```

`PIN` on an `AIN` pin gives volts: ten readings, sorted, the top two and bottom two discarded and the remaining six averaged, scaled 3.3 V over 4095 — MMBasic's filter, constant for constant. On the RP2350B channel n is GP40 + n, so GP41 is channel 1.

One ADC count is about a quarter of a metre over a 1000 m range, so the program ignores movements below two metres. A display that flickers between 141 and 142 is worse than one that is a metre out.

6.27.4 Drawing text on a panel the firmware does not know

This is the part that needs `MM.INFO(FONT ADDRESS)`. The fonts are cached once — address, cell and range, every value read out of the font's own header:

```
FOR f = 1 TO 9
  a = MM.INFO(FONT ADDRESS f)
  fadr(f) = a
  IF a <> 0 THEN
    fwid(f) = PEEK(BYTE a)
    fhgt(f) = PEEK(BYTE a + 1)
    ffst(f) = PEEK(BYTE a + 2)
    fcnt(f) = PEEK(BYTE a + 3)
  ENDIF
NEXT f
```

Drawing one character is then: find its glyph, set the panel's window to exactly one cell, and write the rows. The window is set once and the panel advances by itself, so a glyph costs one address round trip and one write per row rather than one per pixel:

```
g = fadr(f) + 4 + (c - ffst(f)) * w * h \ 8
setwin(x, y, x + w - 1, y + h - 1)
FOR row = 0 TO h - 1
  px$ = ""
  FOR col = 0 TO w - 1
    bidx = row * w + col
    bval = PEEK(BYTE g + bidx \ 8)
    IF (bval >> (7 - (bidx AND 7))) AND 1 THEN
      px$ = px$ + ink$
    ELSE
      px$ = px$ + paper$
    ENDIF
  NEXT col
  SPI WRITE w * 2, px$
NEXT row
```

`ink$` and `paper$` are two-byte RGB565 pixels built once per call, so the inner loop appends rather than converting.

A row is `width × 2` bytes, which is why this works within the 255-byte string limit: 32 bytes for the 16×24 font, 48 for the 24×32. A whole glyph would not fit — the 16×24 one is 768 bytes — but a row always does.

6.27.5 Two traps this example exists to show

The panel does not refuse a window wider than itself. It clamps the window and still accepts every pixel you write into it, so the surplus wraps onto the next row and the character comes out as noise. Fifteen characters of the 16-pixel font is 240 pixels — the whole panel — so a title starting at x=8 loses its last letter into the row below. The program therefore drops a glyph that would overhang rather than drawing it:

```
gx = x + (i - 1) * w
IF gx + w > SW THEN EXIT FOR
```

Silently short is a layout mistake you can see. Silently wrapped looks like a bug in the font reader, and will cost you an evening.

Mid grey is not visible. `RGB565 &H8410` is a perfectly reasonable grey and reads as almost nothing on this panel at 70 % backlight. `&HBDF7` — about three quarters — is legible. Colours that look fine in a table do not always survive a real display, and there is no substitute for looking at it.

6.27.6 What the translation looks like

`MM.INFO(FONT ADDRESS f)` becomes a runtime call and `PEEK` becomes a load, which is the whole point of the exercise — the address is not marshalled anywhere, it is used:

```
v_a = mm_fontaddr(v_f);
H->v_fadr[(int)(v_f)] = v_a;
if (((v_a) != (0LL))) {
    H->v_fwid[(int)(v_f)] = mmpk_byte(v_a);
    H->v_fhgt[(int)(v_f)] = mmpk_byte(((v_a) + (1LL)));
    H->v_ffst[(int)(v_f)] = mmpk_byte(((v_a) + (2LL)));
    H->v_fcnt[(int)(v_f)] = mmpk_byte(((v_a) + (3LL)));
}
```

(`H->` is where arrays and strings live — one allocation the program owns, rather than globals.)

and `mmpk_byte` is one line in `mmb_peek.h`:

```
MMG_FN MMINTEGER mmpk_byte(MMINTEGER addr)
{
    return (MMINTEGER)*MMPK_PTR(unsigned char, addr);
}
```

Reading font data compiles to a `ldrb`. There is no MMU to go through, no copy, and no system call — the flash the kernel keeps its fonts in is simply memory that this program can read.

The whole program is 15 KB of BASIC and compiles to about 63 KB of bytecode.

6.28 Making a big program fit — and run fast

A compiled program pays for its size twice. First there is the obvious budget: a process gets about 330 KB, `bcrun` and its working space take about 170 KB of that, and what remains holds your code (which expands about 1.8× when it is translated to native ARM), your variables and your stack. Second, and less obvious: any single `Sub`, `Function` or main line whose bytecode exceeds the translator's buffers stays **interpreted**, which runs about 2.7× slower — and the main line, the part between the top of the file and the first `Sub`, is compiled as one function. A game whose whole loop lives in the main line can grow that one function past the cap and end up with its hottest code being its slowest.

Recursion goes 255 levels deep. The limit is not the stack: a by-reference argument occupies a slot that cannot be released until the call returns, and there are 256 of them. Past that a program stops with `Expression too complex` rather than misbehaving. For scale, a recursive walk over a million-node balanced tree is twenty levels deep. The guard does not cover routines the translator has turned into native code, so a routine that recurses with no terminating condition is still a program that crashes the machine.

The rules below come from porting PicoMan (Geoff Graham's PacMan, 1,250 lines, graphics-heavy). As it arrived it compiled to 101 KB of bytecode with a 63 KB main line — too big to translate, too big to load fully native. With the compiler taught the rule below it is 38 KB, every function translates, and the whole game images at 134 KB — unmodified. Numbers below are from that exercise. Compute-heavy programs rarely hit any of this — the solar eclipse benchmark has never been near a limit — because number crunching lives naturally in functions over a few variables. Games are different: they draw, and they grow main lines.

1. `ON ERROR IGNORE` is the most expensive statement in the language. For an error to be *survivable*, the compiler must emit checked arithmetic — every array index, every division — so the error is reported rather than corrupting memory. `ON ERROR IGNORE` arms that for an unbounded stretch of the program, so its presence anywhere makes the **whole program** pay: about 2.6× on code size (it was 62 of PicoMan's 101 KB while the compiler still treated every `ON ERROR` this way). `ON ERROR SKIP [nn]`, by contrast, covers only the next `nn` statements, and the compiler emits the checked forms for exactly that window — PicoMan's one `On Error Skip` over a first-time `Blit Close` costs a few dozen bytes, not sixty thousand. So: prefer `SKIP` with a literal count, tightly around the fallible statement, and keep `IGNORE` for short probe programs. Two prints of `MM.ERRNO` cost less than one `IGNORE`. One care with `SKIP` in a compiled program: the interpreter's count follows execution into a called SUB, statement by statement, while the compiled window covers the statements *written* after the `ON ERROR SKIP` (plus one for each routine entered) — a skip that relies on being consumed deep inside a callee's arithmetic behaves like the interpreter only under `IGNORE`. Runtime command errors — a `Blit Close` on nothing, a missing file, an I2C device that does not answer — are trapped anywhere the count is armed, in either world.

2. Keep the main line thin. The main line is one function, and a function only runs at full speed if it translates. Put the work in `Sub`s and `Function`s and keep the main line to setup and a loop that calls them. A web of `GOTO`s pins code into the main line; `GOTO` is fine within a routine, but reaching a block only by `GOTO` keeps it — and everything it drags in — in the one function that must stay small.

3. Locals are cheaper than globals. Every reference to a global variable costs a 5-byte address in bytecode and 8-10 bytes translated; a `Local` or a parameter costs 2-3 and often 2-4. Before its rework, 36% of PicoMan's main line was global addressing. In a `Sub`, declare scratch variables `Local`; pass coordinates as parameters rather than parking them in globals.

4. Fold repetition into a `Sub`. Drawing statements are the widest statements in the language — every argument is a 64-bit expression that must be built and pushed, so one seven-argument `Circle` costs 120-140 bytes *each time it appears*. Four near-identical `Triangle` statements for four directions are one `Sub` with parameters. This is ordinary factoring, but on this machine it is also how the code stays under the translation caps.

5. `DATA` is nearly free; code is dear. `DATA` values go to the data segment byte-for-byte (unused columns are pruned), and reading them is cheap. A table beats a computed unroll: PicoMan's 42 KB shortest-path database costs almost nothing in code, while the same information as `If` chains would be enormous.

Seeing where you stand. `ls -l prog.bc` gives the object size. `THUMB_VERBOSE=1 cc prog.c` prints one line per function — `native: name (bytecode -> native bytes)` — and, for any function that stayed bytecode, the reason: `size policy` means it was too big to translate, and that function will run interpreted. A program that is still too big to load can be built `BCODE_ONLY=1 cc prog.c`: about 2.7× slower, but roughly a third the code.

7 `mmedit`: MMBasic's editor

MMBasic's full-screen editor is ported and runs as an ordinary Fuzix program, so BASIC is written, translated, compiled and run without leaving the machine:

```
# mmedit prog.bas
```

It is the editor from the firmware, with the same keys:

Key	
`ESC`	leave the editor
`F1`	save
`F2`	save and exit (so a wrapper can run the program)
`F3` / `F6`	find / find again
`F4`	mark — then the cursor keys extend the selection
`F5`	paste
`F7` / `F8`	replace / replace again
`F9` / `F10`	read a file in / write the selection out
`F12` or `Ctrl-A`	beautify: re-indent and case the keywords

In mark mode the legend changes: `DEL` deletes, `F4` cuts, `F5` copies, `F10` exports the selection to a file. The status line shows line, column and INS/OVR, and the function key legend shortens itself to fit the terminal width.

The colour coding answers "will this compile?" A keyword `mmbc` can translate is cyan; one only the interpreter knows is blue. That second colour is the useful one here, because a program is compiled rather than run as you type it, and it is worth seeing `MM.WATCHDOG` or `REDIM` in blue before you build rather than after. The lists come from the translator's own tables, so they cannot drift from what it does.

The editor works over the serial port as well as on the HDMI console — it drives a VT100, and the console emits VT100 function key sequences. Files with CRLF line endings are accepted; the CRs are stripped on load, which matters because most files arrive from a PC.

It takes the screen back if a program left it in a graphics mode. The editor draws on the text console, which is what `MODE 1` selects. A program that finished in `MODE 2` leaves the screen 320×240 in sixteen colours, and the console is then not what the monitor is showing — so the editor would be painting where nothing can be seen. `mmedit` switches to `MODE 1` on the way in and puts the old mode back on the way out, including when it is killed, so a program's screen survives a trip through the editor and comes back as it was.

7.1 The other editor: `vi`

`mmedit` is for BASIC. For small text files — a shell script, `/etc/motd`, a short C file — there is a `vi`:

```
# vi hello.c
```

It is Levee, David Parsons' small vi clone, with modal editing, the `:` commands and the usual movement keys. It has been on the card since the beginning under its own name, `levee`, which still works: `vi` and `levee` are two names for one file.

Its edit buffer is 64 KB, which is enough for any source file you are likely to write on the machine. Levee's own default is 4 KB — it was written for 8-bit micros, where that was a fair share of the machine — and this port raises it, since a process here may have most of a 340 KB pool to itself.

Open something larger than the buffer and Levee says `[overflow]` on the status line and holds the file **read-only**: you can look and move around, but `:w` answers "File is readonly" rather than writing a truncated file back. That is the safe way round, and worth knowing as the signal that a file is too big rather than that something is wrong.

One difference from a full `vi` to watch: `cw` on the last word of a line joins the following line onto it.

8 The FAT partition and the `fat` command

Partition 1 of the SD card is ordinary FAT, readable and writable by any PC — this is how files travel to Fuzix, because nothing else can mount the Unix partition. Format it once in Windows (FAT or FAT32, **not exFAT**), then simply copy files onto it.

On the Fuzix side, the `fat` command reads it (long filenames and subdirectories included):

```
# fat info
/dev/hdb1: FAT16, 32695 clusters of 2048 bytes (62 MB)
# fat ls
    4523  My Program.bas
   <dir>  BASIC
# fat ls basic
# fat get "my program.bas" /root/prog.bas
# fat get demo.bin                    (lowercased name in current dir)
```

`fat` is read-only by design — the desktop does the writing. To move a file *out* of Fuzix, use the serial port, or write it with `doswrite` (the venerable Minix FAT12/16 tool, also included) if the partition is FAT16.

9 Moving files over the serial port

The FAT partition carries files *in*, but `fat` is read-only, so it cannot carry anything back *out*. The serial console can do both, in either direction, with nothing on the board but `uue` and `uud` — and it is the only route that needs no card swapping.

The idea is older than the machine: `uuencode` turns any file into printable text, and printable text survives a terminal. Both tools are in `/bin`:

```
# uue FILE            writes FILE.uue - the encoded text
# uud FILE.uue        decodes it back, recreating the original name
```

9.1 Sending a file to the board

On the PC, encode the file, then have the board read the text into a file and decode it:

```
# cat > prog.uue          (now paste or send the encoded text)
^D
# uud prog.uue
# ls -l prog.bas
```

The one thing that will bite you is flow control. The terminal input queue is 132 bytes. Sending encoded text as fast as the port will take it overruns that queue and loses characters silently — a 62-character line sent every 25 ms still loses roughly 60% of the text, and the damage only shows up as a corrupt file at the end. Two ways to avoid it:

- **Let the terminal pace it.** Any sender with a per-line delay *and* a wait for the echo will do. The delay alone is not enough.
- **Use the supplied script.** `devtools/uusend.py` in the source tree does exactly this: it types the text one line at a time and waits for each line to echo before sending the next, so at most one line is ever in flight. It then runs `uud` for you.

```
python uusend.py local.bas prog.bas --port=COM11
```

Note that `uusend.py` takes a **bare** remote filename — it drives one `ucp`-style session with `cd` and plain names, so `mv` the file afterwards if it belongs somewhere else.

9.2 Fetching a file from the board

The other direction is easier, because the board is the slow end:

```
# uue report.txt
# cat report.uue
```

Capture the session to a file in your terminal program, cut away anything before `begin` and after `end`, and decode it on the PC.

9.3 Programs to use

Linux (and macOS, and WSL):

- `uuencode` / `uudecode` come from the **sharutils** package (`apt install sharutils`). macOS has `uuencode` built in.
- For the terminal, **picocom** (simplest), **minicom** (its `Ctrl-A S` menu includes an `ascii-xfer` that paces lines), or **screen /dev/ttyUSB0 115200**.

- If sharutils is awkward, Python needs no install: `python3 -c "import binascii,sys; ..."` — or just use `devtools/uusend.py`, which does the encoding itself.

Windows:

- **Tera Term** is the pick — free, and its *File → Send file* has a per-line delay setting under *Setup → Serial port* (set a transmit delay of a few ms per character or 30–50 ms per line). This is the easiest reliable route without scripting.
- **PuTTY** works for capture (*Session → Logging*) but has no paced file send; pasting a large file into it will lose characters.
- **RealTerm** can send a file with a configurable delay, and is good at capturing binary.
- For the encode/decode step, Windows has no built-in `uuencode`. Use **WSL** (then it is the Linux instructions), **Python** for Windows, or **UUDeview**, which still builds and runs.

Whatever you use, the check at the end is the same: `sum` the file on both machines and compare. On the board, `sum FILE`.

10 Commands at the # prompt

The shell is the Bourne shell and the machine underneath it is a V7-style Unix, so most of what you know already works. This chapter is about the rest: the programs that exist because this is a Pico Computer rather than a PDP-11, and the handful of places where the tools have been brought forward from 1979.

Everything named here is on the card. That is checked rather than promised — `mancheck.sh` extracts every command name from this chapter and the next and looks for it in the built image, and the release will not go out if one is missing. The list used to be maintained by hand and had drifted badly: it offered a games collection of some thirty titles, an editor called `ue`, and an assembler, none of which were ever installed.

10.1 The machine itself: `picoctl`

Everything the kernel will do on request that has no better home.

```
# picoctl flash                 reboot into the UF2 bootloader
# picoctl keymap uk             us uk de fr es be
# picoctl mode 2                set the display mode
# picoctl mode                  ... and back to the 80x40 text console
# picoctl usbreset              re-enumerate after the DPDT switch
# picoctl numlock               report
# picoctl numlock off           for the keyboard plugged in now
# picoctl numlock on 04b3:3025  for one that is not
# picoctl numlock off --once    change it without saving
# picoctl numlock --load        replay the saved settings (/etc/rc does this)
```

`picoctl flash` is the polite way to reach the bootloader: `sync`, `remount -n / ro`, `sync`, then `picoctl flash`, and the drive appears without anyone touching the board.

Num lock is a property of the keyboard, not of the machine. A keyboard with no numeric keypad usually overlays one onto `7890/uiop/jkl;/m` while num lock is on, and its own firmware does that from the LED report the host sends. Nothing in the USB descriptors distinguishes such a keyboard — a Raspberry Pi keyboard and a full-size Lenovo return byte-identical HID report descriptors — so the setting is remembered per keyboard in `/etc/numlock`, written when you change it and replayed by `/etc/rc`. That is why it survives a reboot and why each of your keyboards keeps its own answer.

10.2 Pictures

These four are the programs `SAVE IMAGE`, `LOAD IMAGE`, `LOAD JPG` and `LOAD PNG` run, and they are just as useful typed at the prompt. All of them want a graphics mode — `picoctl mode 2` if you are at the text console.

```
# saveimage shot.bmp                    the whole screen
# saveimage part.bmp 160 120 320 240    x y w h
# loadimage shot.bmp                    at 0,0
# loadimage tiger.bmp 0 0 4             dither mode 4
# loadjpg photo.jpg                     at 0,0
# loadjpg photo.jpg 0 0 0 0 0 2         binned 2:1, so 640x480 fits
# loadpng logo.png 40 20                at x,y
# loadpng logo.png 0 0 -1               let the screen show through
```

```
# loadpng -s logo.png > logo.spr         decode to a SPRITE, on stdout
```

The full argument lists, which are MMBasic's:

```
saveimage  file.bmp [x y w h]
loadimage  file.bmp [x [y [mode [ximage [yimage]]]]]
loadjpg    file.jpg [x [y [mode [ximage [yimage [scale]]]]]]
loadpng    file.png [x [y [transparent [cutoff]]]]
```

`ximage`/`yimage` are the offset **into the picture**, so a large image can be cropped rather than scaled. `saveimage` writes a 24-bit BMP of any rectangle. `loadimage` is MMBasic's own BMP decoder and reads 1/4/8/16/24/32-bit files, `BI_BITFIELDS` and RLE4/RLE8, dithering only when asked.

`transparent` defaults to 0, which is palette black, so a PNG with a transparent surround draws a black box on a white screen. That looks like a bug and is not — a real PicoMite does the same, and the default is chosen so that `SPRITE LOADPNG` and `SPRITE SHOW`, which both treat index 0 as see-through, meet in the middle. Pass **`-1`** to let the screen show through instead. `cutoff` is the alpha above which a pixel counts as opaque.

`loadjpg` decodes a block at a time, so its memory does not grow with the picture. `loadpng` cannot work that way — PNG filters refer to the row above and the whole image must be inflated first — so it takes the big buffers from the PSRAM arena and costs the process pool nothing.

The `-s` form needs no graphics mode at all: it writes the decoded sprite to standard output as two 16-bit sizes followed by one colour index per byte. That is how `SPRITE LOADPNG` works, and it is also the way to convert artwork from the shell.

10.3 Sound

```
# playmp3 track.mp3            volume 80 by default
# playmp3 track.mp3 40         0 to 100
# playmp3 track.mp3 &          in the background, and carry on working
# playwav clip.wav 60
# playflac album.flac
# playmod tune.mod 70          MOD, and it loops
# playmod tune.mod 70 noloop   ... or plays once
```

```
playmp3, playwav, playflac  file [volume 0-100]
playmod                     file.mod [volume] [noloop]
```

These are what `PLAY MP3`, `PLAY WAV`, `PLAY FLAC` and `PLAY MODFILE` run. **One plays at a time**: a second is refused by name and pid rather than allowed to talk over the first. Stop one with `kill -2`, or `PLAY STOP` from BASIC. `playmp3` decodes at about six times real time and leaves most of the machine to whatever else is running.

`PLAY SOUND` and `PLAY TONE` have no command-line equivalent — the synthesiser lives in the kernel's DAC interrupt and is reached through `/dev/sys`.

10.4 Languages and the toolchain

Each of these has a chapter of its own; this is the summary.

```
# cc prog.c                     -> prog.bc, and run it with ./prog.bc
# cc prog.bas                   BASIC in one step, via mmbc
# mmbc prog.bas                 -> prog.c, and stop there
# bcdump prog.bc                disassemble the bytecode
# bbcbasic                      BBC BASIC, with its own editor
# fforth                        a complete ANS Forth
# dc                            the V7 desk calculator
```

```
cc      [-o out] [-v] [-k] [-Ldir] file.c \| file.bas
mmbc    source.bas [-o out.c] [--report] [--strict] [--fcc]
bcdump  file.bc
```

`cc prog.bas` is the whole build in one command: it runs `mmbc` first and writes `prog.mb.c`, a deliberately different name from `prog.c` so that compiling `prog.bas` can never overwrite C you wrote yourself. That file is deleted again with the other intermediates, so the shortcut leaves no C behind: pass `-k` to keep it, or run `mmbc` on its own when the generated C is what you are after. `mmbc --report` lists implied globals and any line it could not translate; `--strict` stops on the first of them instead of commenting it out and carrying on.

`bcrun` executes a `.bc` file, and is what the `#!`-less `./prog.bc` actually invokes.

10.5 Editors

```
# mmedit prog.bas         MMBasic's own, with its function keys
# vi hello.c              levee, a compact vi
# ed                      the V7 line editor, still here
```

`mmedit` is described in its own chapter. `vi` and `levee` are two names for one program.

10.6 Getting files on and off

```
# fat ls                        list the FAT partition
# fat ls basic                  ... a directory of it
# fat get "my program.bas" /root/prog.bas
# fat info
# uue report.txt                -> report.uue, to paste into a terminal
# uud prog.uue                  decode, recreating the original name
# rx file                       XMODEM in, over /dev/tty2
# sx file                       ... and out
# dosread                       FAT12/16 floppy-era transfers
# tar                           the classic archiver
```

`fat` takes `[-d device]` before its subcommand. The FAT partition is the one Windows can see, so it is the easy way to move a file from a PC; `uue`/`uud` need nothing but the console, and have a chapter of their own.

10.7 The clock, and housekeeping

```
# setdate                       show the time
# setdate -w                    write the system time to the DS3231
# setdate -a                    ask for a new one
# df                            space
# free                          memory, and what the swapper took
# ps                            processes
# uptime                        how long since the last boot
# mount                         what is mounted
# umount /dev/hdb3              ... and unmount it
# remount -n / ro               read-only, before flashing
# fsck /dev/hdb2                check a filesystem
# sync                          flush the buffers
# shutdown                      the tidy way to stop
# halt                          ... and the blunt one
# reboot                        start again
```

`setdate` also takes `-u` for UTC and `-0` to zero the clock. `swapon` exists but has nothing to do: there is no swap device any more, because the kernel takes an allocation the size of the process out of PSRAM when it needs to swap one out. `free` reports that.

`setboot`, `prtroot` and `substroot` choose and report which partition the kernel boots from — see Appendix B.

10.8 Where this is not 1979

Fuzix is a V7-style Unix and most of the userland is period-correct. These are the places it is deliberately not, and they are worth knowing because they are what makes shell work on the machine bearable.

- **`awk` is Lucent's one true awk** — the maintained descendant of the V7 original, by one of its authors, not a subset. Associative arrays, user-defined functions, `getline` in each of its forms, output pipes, dynamic regular expressions, `printf` and the maths library. See Text processing.
- **The C library can print a fraction.** `printf("%f", x)` used to print the letter `f`. `%e`, `%f` and `%g` work now, in `printf` and `scanf`, and so do `%j` and `%z`. A bytecode program's `printf` gained `%e` and `%g` too, where before they printed themselves *and* misaligned every argument after them.
- **`[` exists.** `/bin/test` was there and `/bin/[` was not, so every bracket test in every shell script failed with `[: not found`.
- **`sed`, `find`, `expr`, `seq` and `uname`** were all being compiled and simply never installed. They are installed.
- **`grep -q`, `ls -t` and `ls -1`** — small additions, and the ones a script reaches for first.
- **`mmedit`** is not a Unix tool at all: it is MMBasic's own full-screen editor, keyword colouring and function keys included.

What is still period-correct, so that a script does not surprise you: `grep` has basic regular expressions only and no `-E`; `sed` is MINIX's, with no `-E`; `sort` uses the V7 key syntax `+beg -end` rather than `-k`; `diff` has no `-u`. The table in Text processing has the detail.

10.9 What is not here

A short list, because a manual that names a command the card does not have wastes more of your time than one that stays quiet.

There is no `less`, `make`, `m4`, `factor`, `units` or `mail`, and no assembler. `/usr/games` holds `advent` — the original Colossal Cave — and `cowsay`, and that is all: the Scott Adams, Mysterious Adventures and Z-machine collections are not installed. There is no network, so `ssh` cannot do anything useful. `man` is present and so are its pages, in `/usr/man/man1`.

ewpage

11 The filesystem and included software

11.1 Layout

```
/bin          core utilities, and picoctl, fat, uue/uud, setdate
/usr/bin      the toolchain, BBC BASIC, the editors, the media tools
/usr/games    advent and cowsay
/usr/lib/cc   the C compiler passes, and its headers in include/
/usr/man      manual pages (man <name>)
/dev          devices - see below
/etc          rc (boot script), inittab, passwd, termcap
/tmp          scratch space
/root         root's home directory
```

Key devices:

Device	What it is
`/dev/tty1`	The console (HDMI + USB keyboard + serial mirror)
`/dev/tty2`	The GP0/GP1 serial port
`/dev/hda`	On-board flash filesystem, 15 MB
`/dev/hdb1`–`hdb3`	SD card partitions (root is `hdb2`)
`/dev/rtc`	The DS3231 clock (`setdate` reads and sets it)
`/dev/sys`	Platform control (graphics, sound, ADVAL ioctls)

`/etc/rc` runs at boot: filesystem check, clock from the DS3231, keyboard layout. Edit it with any of the editors below.

There is no swap device to enable. The PSRAM is not a block device any more: the kernel takes an allocation the size of the process when it needs to swap one out, and programs take their heap from the same place. `free` reports both.

11.2 Applications

The programs that make this machine what it is have a chapter of their own: Commands at the `#` prompt covers `picoctl`, the image and sound players, the toolchain, the editors and the file-transfer tools, with their arguments.

Shell and core: Bourne `sh` and the classic set — `ls ll cp mv ln rm mkdir rmdir cat more head tail grep fgrep sed awk tr cut sort uniq wc find xargs diff diff3 cmp comm join split rev tar dd df du free ps kill killall uptime date cal banner echo sleep tee touch which who su passwd stty mount umount sync fsck mkfs fdisk chmod chown chgrp od seq dc expr test uname cron at write wall` — and more, so `ls /bin /usr/bin` is the authority rather than this paragraph.

Games (`/usr/games`): `advent`, the original Colossal Cave, and `cowsay`.

11.3 Text processing

`awk` is Lucent's — the one true awk, the maintained descendant of the V7 original by one of its authors. It is the real thing rather than a subset: associative arrays, user-defined functions, `getline` in each of its forms, output pipes, dynamic regular expressions, `printf`, and the maths library.

```
# awk '{ s[$1] += $2 } END { for (k in s) printf "%-10s %6.2f\n", k, s[k] }' log
# ps | awk 'NR > 1 { n++ } END { print n " processes" }'
```

With `sed`, `grep` and `find` alongside it the usual pipelines work:

```
# find /bin -name 'a*' | sed 's|/bin/||' | awk '{ print NR, $1 }'
```

What these are not. They are period tools and they show it in places, so it is worth knowing where before a script surprises you:

`grep`	basic regular expressions only. `-c -e -i -l -n -q -s -v`, and no `-E`; use `awk` for an extended regex, or `fgrep` for a fixed string.
`sed`	MINIX's. Substitution, `-n`, several `-e`, and back-references all work; no `-E`.
`sort`	the V7 key syntax, `+beg_pos -end_pos`, not `-k`.
`diff`	V7 — no `-u`, so no unified diffs.
`ls`	`-t` sorts a *directory* by time, newest first; command-line arguments are listed in the order you gave them, as they always have been.
`test`	and `[`, which are one program under two names. `-a -b -c -d -e -f -g -n -o -p -r -s -t -u -w -x -z`.

12 Appendix A: pin usage

GPIO	Function
GP0 / GP1	Serial port `/dev/tty2` (TX / RX)
GP8 / GP9	System console UART (USB-C CH340)
GP10/11/22	Audio I2S: BCLK / LRCLK / DATA (PCM5102 DAC)
GP12–GP19	HDMI (HSTX)
GP20 / GP21	I2C0: DS3231 RTC and QWIIC connector (SDA / SCL)
GP27	DS3231 32 kHz — board identification (PC3 only)
GP28/30/31/33	SD card, Pico Computer 3 (MISO/SCK/MOSI/CS, SPI1)
GP29/30/31/32	SD card, Pico Computer 2 (CS/SCK/MOSI/MISO, bit-banged)
GP32	DS3231 alarm interrupt (PC3; unused by Fuzix)
GP34–GP37	Joystick switches — `ADVAL(0)` bits 0–3
GP40	Analogue input (reserved; not read by ADVAL)
GP41–GP44	Analogue inputs — `ADVAL(1)`–`ADVAL(4)`
GP45/GP46	Analogue-capable, free
GP23/24/25/29	CYW43 wireless (PC3; unused by Fuzix)
GP47	PSRAM chip select

All spare header pins remain ordinary 3.3 V GPIO. Do not apply 5 V to any GPIO.

13 Appendix B: boot options

The kernel command line (held by the bootloader) accepts:

- `hda` / `hdb2` ... — root device override
- `kbd=us|uk|de|fr|es|be` — early keyboard layout override (the layout in `/etc/rc` then applies for the session)

The build, source and development notes live in the `pc3` branch of `github.com/UKTailwind/FUZIX` under `Kernel/platform/platform-rpipico/` — see `PC3-DEVNOTES.md` for the engineering history and `PC3-GFX-DESIGN.md` for the display design.

14 Appendix C: MMBasic coverage

This is what `mmbc` translates today. It is generated from the translator's own tables (`fcc/coverage.py` in the mmb2c repository), so it says what the program does rather than what anyone remembers it doing.

Coverage grows with each release, and it will never be complete. MMBasic is a large language whose statements reach deep into one particular firmware, and a translator that emits portable C cannot follow all of it. Anything not listed here is reported by name, with its line number, and the translation continues - so you find out at translate time, not at run time.

14.1 Statements

?	ARC	ARRAY	BEZIER
BIT	BLIT	BOX	BYTE
CALL	CASE	CAT	CHDIR
CIRCLE	CLEAR	CLOSE	CLS
COLOR	COLOUR	CONST	CONTINUE
COPY	DATA	DATE$	DEFINEFONT
DIM	DO	ELSE	ELSEIF
END	ENDIF	ERASE	ERROR
EXIT	FILES	FILL	FLAG
FLAGS	FLASH	FLUSH	FONT
FOR	FRAMEBUFFER	FUNCTION	GOSUB
GOTO	GUI	I2C	I2C2
IF	INC	INPUT	KILL
LET	LINE	LMID	LOAD
LOCAL	LONGSTRING	LOOP	MAP
MATH	MKDIR	MODE	NEXT
ON	ONEWIRE	OPEN	OPTION
PAUSE	PIN	PIXEL	PLAY
POKE	POLYGON	PORT	PRINT
PULSE	PWM	RANDOMIZE	RBOX
READ	REDIM	RENAME	RESTORE
RETURN	RMDIR	RTC	SAVE
SEEK	SELECT	SERVO	SETPIN
SETTICK	SORT	SPI	SPRITE
STATIC	STRUCT	SUB	SYSTEM
TEMPR	TEXT	TIME$	TIMER
TRIANGLE	TYPE	WEND	WHILE

Assignment needs no keyword (`LET` is accepted). Statement separators, line numbers and labels, `REM` and `'` comments all work as expected.

14.2 Functions

ABS	ACOS	ASC	ASIN
ATAN2	ATN	BIN$	BIN2STR$
BIT	BOUND	BYTE	CHOICE

CHR$	CINT	COS	CWD$
DATE$	DATETIME$	DAY$	DEG
DIR$	EOF	EPOCH	EXP
FIELD$	FIX	FLAG	FORMAT$
HEX$	INKEY$	INPUT$	INSTR
INT	KEYDOWN	LCASE$	LCOMPARE
LEFT$	LEN	LGETBYTE	LGETSTR$
LINPUT	LINSTR	LLEN	LOC
LOF	LOG	LTRIM$	MAP
MATH	MAX	MID$	MIN
MM.CMDLINE$	MM.DEVICE$	MM.ERRMSG$	MM.ERRNO
MM.FONTHEIGHT	MM.FONTWIDTH	MM.HPOS	MM.HRES
MM.I2C	MM.INFO	MM.INFO$	MM.ONEWIRE
MM.SPISPEED	MM.VER	MM.VPOS	MM.VRES
OCT$	PEEK	PI	PIN
PIXEL	PORT	POS	RAD
RGB	RIGHT$	RND	RTRIM$
SGN	SIN	SPACE$	SPI
SPRITE	SQR	STR$	STR2BIN
STRING$	STRUCT	TAB	TAN
TEMPR	TIME$	TIMER	TRIM$
UCASE$	VAL		

14.3 MATH() sub-functions

Scalar: `ATAN3`, `COSH`, `LOG10`, `SINH`, `TANH`

Whole-array (one number out of an array): `MAX`, `MEAN`, `MEDIAN`, `MIN`, `SD`, `SUM`

14.4 MATH sub-commands

`MATH` is also a statement, and that is a different and much longer list in the interpreter. Six of it are translated — the ones that walk an array element by element, which is what most programs use it for:

`MATH SET v, a()`	every element of `a()` becomes `v`
`MATH ADD a(), v, b()`	`b() = a() + v`, element by element
`MATH SCALE a(), v, b()`	`b() = a() * v`, element by element
`MATH RANDOMIZE [seed]`	seed the generator; no seed uses the clock
`MATH SLICE a(), i, , k, b()`	copy one line of `a()` into `b()`
`MATH INSERT a(), i, , k, b()`	copy `b()` back into that line

`SET`, `ADD`, `SLICE` and `INSERT` take integer, float or string arrays; `SCALE` is numeric only, as it is there. `ARRAY` is accepted as a spelling of `MATH` for all of them, and `ARRAY SLICE` and `ARRAY INSERT` are the spellings the manual uses — the interpreter runs both through the same two functions.

14.4.1 Slicing an array

A **slice** is one line through an array of two or more dimensions. Give every index but one; the line runs along the one left blank, and the array it is copied to or from is one-dimensional and the same length.

```
DIM a(3, 4, 5), b(4)
ARRAY SLICE  a(), 2, , 3, b()    ' b() = a(2,0,3), a(2,1,3), ... a(2,4,3)
b(0) = 99
ARRAY INSERT a(), 2, , 3, b()    ' and back again
```

It is far quicker than the equivalent `FOR` loop, and that is the reason it exists: the elements of the line are a fixed distance apart, so the copy is a single stride and the index arithmetic is done once at translate time rather than once per element. `OPTION BASE` is respected on both sides. A length that does not match is `Size mismatch between slice and target array`, as in the interpreter.

One difference to know about: MMBasic converts between integer and float arrays here, and this does not — both arrays must be the same type, the same rule `ARRAY ADD` already follows.

The rest are not translated, and each says so by name rather than being mistaken for something else: the matrix operations (`M_MULT`, `M_INVERSE`, `M_TRANSPOSE`, `M_PRINT`), the vector ones (`V_MULT`, `V_CROSS`, `V_NORMALISE`, `V_ROTATE`, `V_PRINT`), the quaternions (`Q_CREATE`, `Q_EULER`, `Q_INVERT`, `Q_MULT`, `Q_ROTATE`, `Q_VECTOR`), the complex arithmetic (`C_ADD`, `C_SUB`, `C_MUL`, `C_DIV`, `C_AND`, `C_OR`, `C_XOR`), `FFT`, `WINDOW`, `SINC`, `INTERPOLATE`, `POWER`, `SHIFT`, `PID`, `SENSORFUSION` and `AES128`.

They are a coherent block of work rather than a scattering of gaps — most are pure arithmetic over arrays with no hardware in them, so they would go in as a header of static functions and cost nothing to a program that does not use them.

14.5 Types and structure

`INTEGER` (64-bit), `FLOAT` (double), `STRING`, and arrays of each, up to the dimensions MMBasic allows. `DIM`, `LOCAL`, `STATIC`, `CONST`, `OPTION BASE`, `SUB` and `FUNCTION` with by-reference arguments, and the usual control flow.

`DIM s$(n) LENGTH m` sets the SPACING of the elements, not just a cap on each. The firmware places element `k` at `base + k * (m + 1)`, and a program is entitled to that: PETSCII Robots keeps its 128x64 world map as `DIM LV$(63) LENGTH 128` and reads tiles straight out of it with `PEEK(BYTE (y) * 129 + x + lva)`, where the 129 *is* the `LENGTH`. So an array honours it, and an element is then the firmware's own layout — a length byte and `LENGTH` characters, with no room for the trailing NUL the runtime normally keeps. That is handled for you; the one visible consequence is that such an array cannot be passed whole to a `SUB` or to `SORT`, because the runtime's array helpers step at the full string size. Index it instead, or leave the `LENGTH` off. On a *scalar* `LENGTH` is still only a cap, since a scalar's layout cannot differ.

Structures (MMBasic's `TYPE ... END TYPE`, documented in full in the PicoMite structures manual) are translated with the firmware's byte layout reproduced exactly — `STRUCT(SIZEOF "t")` answers the same number here and on a PicoMite. Members may be `INTEGER`, `INT`, `FLOAT`, `STRING [LENGTH n]`, arrays of those, or an earlier TYPE; member access works to the full nesting depth (`data(2).items(1).values(4)` included), structure variables, arrays, `LOCAL`s and by-reference parameters all work, whole structures assign with `=`, and `STRUCT COPY`, `STRUCT CLEAR` and `STRUCT SWAP` are in. `STRUCT(SIZEOF/OFFSET/TYPE)` fold to constants when the names are literal strings.

Not translated (each says so rather than mistranslating): `STRUCT SORT/SAVE/LOAD/PRINT/EXTRACT/INSERT`, `STRUCT(FIND)`, a `FUNCTION` returning a structure, whole structure arrays as parameters, and initialisers on structure arrays. Assigning a whole structure into or out of a *nested* member is refused deliberately: the interpreter copies the outer type's size there and overruns memory, and a clean error beats reproducing that.

14.6 Errors, and surviving them

In a program that uses `ON ERROR`, every error it can hit is a check the runtime makes *before* it does the thing — the same order the interpreter uses. Nothing is left to the hardware to notice: dividing by zero as a float would otherwise answer `inf` rather than stopping, and this processor does not trap integer division by zero at all.

A program with no `ON ERROR` anywhere is compiled without the arithmetic checks — see "What `ON ERROR` costs" below. Its float divisions and `SQR`/`LOG`/`ASIN`/`ACOS` take what C gives: `inf` or `nan`, flowing onward through the sums. There is no sensible recovery from arithmetic like that — the program has a bug to fix either way — so the choice is between stopping with a message (a trapping program) and full speed (everything else). Integer division and `MOD` are the exception, checked in every program: C leaves integer division by zero undefined, and the silent zero this processor answers is not a number to limp onward with.

`ON ERROR SKIP [n]`, `ON ERROR IGNORE`, `ON ERROR CLEAR` and `ON ERROR ABORT` work as they do on a PicoMite, with `MM.ERRNO` and `MM.ERRMSG$` reporting what happened. A statement that fails while an error is being skipped is abandoned where it failed: the assignment does not happen, the rest of the `PRINT` does not print, and execution resumes at the next statement — in the `SUB` it happened in, if that is where it was. The count works as it does there too, `SKIP n` covering the next n statements.

Two details that surprise people, both matching the interpreter. A `PRINT` that fails part way through prints **nothing at all** — not even the items before the failure — because the line is built whole and the error discards it. And calling a `SUB` spends skip count: the `SUB` line and any `LOCAL` are statements too, so `ON ERROR SKIP 2` before a call does not reach the third statement inside it.

Two limits worth knowing. A statement that jumps away (`GOTO`, `EXIT`) does not count against `SKIP n`, so a count that spans one runs one statement further than it would on a PicoMite. And running out of memory ends the program whatever `ON ERROR` says — there is nothing sensible to carry on with, and a program limping along on a failed allocation would fail somewhere far less obvious.

`ON ERROR RESTART` reboots the machine on a PicoMite; a compiled program has no equivalent, so `mmbc` refuses it by name rather than guessing.

14.6.1 What `ON ERROR` costs

The checks are real work in real loops. With `ON ERROR` present, every float division whose divisor is not a nonzero literal becomes a runtime call that tests the divisor first, and `SQR`, `LOG`, `ASIN` and `ACOS` go through checked wrappers instead of straight to libm — that is what makes their errors trappable statements. Measured on the machine from a fresh boot: the grains benchmark, whose inner loop divides by a variable and takes a logarithm every pass, pays about 12%; the solar eclipse predictor, dominated by plain arithmetic, about 2%. The string checks (concatenation past 255 characters, `ASC` of an empty string) are part of string calls that happen anyway and stay on in every program; they cost a comparison, not a call.

`ON ERROR` earns its keep trapping things that genuinely can fail at run time — an I2C transfer that may not be acknowledged, a file that may not exist — not arithmetic. If a program traps errors out of habit, leaving `ON ERROR` out buys the checks back.

14.6.2 I2C2 — the second controller

`I2C2` is MMBasic's second bus, and on this machine it is the one a program may have. The fixed bus (`I2C`, GP20/GP21) is the QWIIC socket *and* the DS3231 the system clock runs on, which the kernel

polls from interrupt context, so it stays the kernel's.

```
SETPIN 38, 39, I2C2          ' GP38/GP39 or GP42/GP43 - see below
I2C2 OPEN 400, 1000          ' speed kHz, timeout ms
I2C2 WRITE &H77, 1, 1, &HD0  ' option 1 = HOLD: no STOP
I2C2 READ  &H77, 0, 1, r$    ' ...so this is a repeated START
I2C2 CLOSE
```

The pins are not free to choose: the RP2350 muxes I2C1's SDA only onto pins where `pin AND 3 = 2` and SCL only where `pin AND 3 = 3`, so on this board the pairs are **GP38/GP39** and **GP42/GP43**. Anything else is refused at `OPEN` rather than producing a bus that never answers.

The data arguments are the same on all three buses, which is worth learning once. `WRITE` takes a list of byte expressions, a whole numeric array written `a()`, a string, or a long string written `LONGSTRING a()`. `READ` takes any of those, and also **a list of variables, one per byte received**:

```
I2C2 READ &H77, 0, 3, hi, mid, lo
```

MMBasic reaches one implementation of these from `I2C`, `SPI` and `ONEWIRE` alike, and so does this — learn them here and they work there. The string forms are what MMBasic's own sensor examples use, because `STR2BIN()` then lifts the 16- and 32-bit fields straight out of what was read.

Two rules the shared code enforces, both MMBasic's. A list must have as many values as the count says, or you get `Argument count` — the count is not a way to send fewer. And a destination is checked *before* the transfer, so asking for forty bytes into an array of sixteen is refused without the bus moving: a read addresses a device and can advance a register pointer inside it, so it must not happen and then fail.

Option bit 0 **holds** the bus: the transfer ends without a STOP so the next one is a repeated START on the same device. Most register-file devices do not need it — they keep their address pointer across a STOP — but the ones that do would otherwise read plausible nonsense.

Two differences from a PicoMite, both deliberate:

- `timeout` is milliseconds, `0` or `100` and up as MMBasic requires, but **`0` means a five-second cap, not "no timeout"**. This kernel is non-preemptive: a transfer that waited for ever would not hang the program that asked for it, it would hang the machine, console and display included.
- A held bus that is never finished — the program errors, or is killed between the write and the read — is finished by the kernel, which clocks SCL until the device releases SDA and then issues a STOP. Nothing else on the board would free it.

The pins and the controller are claimed through the pin lock, so they come back if the program dies, and a second program asking for the bus gets a clear "already in use" rather than a collision.

14.6.3 SPI — the first controller

```
SETPIN 2, 3, 4, SPI          ' any order: the pin decides its own role
SPI OPEN 62500000, 0, 8      ' speed, mode, bits
SPI WRITE 4, &H2A, 0, 0, 0   ' a list, an array a(), or a string
SPI READ 3, r$               ' into a string or a numeric array
v = SPI(&H55)                ' write one unit, read one back
PRINT MM.SPISPEED            ' the clock it ACTUALLY got
SPI CLOSE
```

The order of the three pins does not matter. The RP2350 decides which signal each one carries: the controller is `(pin AND 8) = 0` for SPI0, and the role is `pin AND 3` — 0 MISO, 1 CS, 2 SCLK, 3

MOSI. So GP2/GP3/GP4 are SCLK/MOSI/MISO however you write them. On the header SPI0 is **GP0–GP7 and GP34–GP39**; the rest (GP26, GP40, GP42–GP46) are SPI1, which is **the SD card** and is not offered — a program that could take it could take the filesystem out from under itself. This is MMBasic's behaviour too: it asks each pin what it can be rather than fixing an order.

Chip select is yours, exactly as on a PicoMite. A display needs CS held across a whole command-and-data sequence rather than per transfer, so only the program knows when to move it — and a pin is a register write now, not a system call.

`MM.SPISPEED` is worth using. The divisor is `clk_peri / (CPSDVSR × (1 + SCR))` with `CPSDVSR` even, so a request nearly always lands on a neighbouring value: asking for 50 MHz gives 46.875, and anything above `clk_peri / 2` quietly becomes `clk_peri / 2`. The usable steps here are 62.5, 46.875, 37.5, 31.25 MHz and down.

A whole transfer is **one system call whatever its length** — the controller reads your buffer where it lies rather than copying it through the kernel, which is sound here because there is no MMU and the kernel cannot be preempted. A 240×320 screen of 16-bit pixels is 153,600 bytes and goes out in 26 ms at 62.5 MHz.

A string holds at most 255 bytes, so a 240-pixel row of RGB565 is 480 and will not fit in one. **A long string will**, and that is what the extra word is for:

```
DIM INTEGER row(70)                ' 560 bytes of payload
' ... fill it with LONGSTRING APPEND ...
SPI WRITE 480, LONGSTRING row()    ' the whole row, one call
SPI READ  480, LONGSTRING row()    ' and back
```

Say `LONGSTRING` and mean it, because a long string *is* an integer array: written `row()` it is a numeric array and sends one byte per eight-byte cell, quietly and wrongly. That is MMBasic's behaviour, and the extra word is how you say which of the two you meant.

It is not a speed feature. Filling 240×150 one call per row against 100 pixels a call measures 30 ms against 32 at 24 MHz, and 12 against 13 at 62.5 — the bus dominates. What changes is that the row can be assembled at all, and with it a whole frame.

14.6.4 One-wire and `TEMPR`

```
ONEWIRE RESET 26                   ' MM.ONEWIRE = 1 if something answered
ONEWIRE WRITE 26, 1, 1, &H33       ' reset first, then READ ROM
ONEWIRE READ  26, 0, 8, id()       ' the eight ROM bytes
PRINT TEMPR(26)                    ' a DS18B20, degrees C
```

Any header pin will do; the slots are timed in the program itself. The flag is MMBasic's: **1** reset first, **2** reset afterwards, **4** send or read single bits rather than bytes, **8** hold a strong pull-up when finished, for a parasite-powered device.

The data and destination arguments are **the same ones `I2C2` and `SPI` take** — a list, a string, a whole numeric array, a long string, or (on a read) a list of variables one per byte. That is not a coincidence: MMBasic reaches the same two functions from all three buses, and so does this.

`TEMPR(pin)` starts a conversion and waits for it. `TEMPR START pin[, precision[, timeout]]` starts one and returns immediately — 6 ms measured — so the program can do something useful during the 750 ms a 12-bit conversion takes, and collect the reading later:

```
TEMPR START 26, 3                    ' 12 bits
' ... do something else ...
t = TEMPR(26)
```

The wait sleeps rather than spins, which is the one place this differs from MMBasic on purpose. MMBasic has one program to run and can afford to spin out three quarters of a second; this machine has others. A `SETTICK` handler keeps firing throughout — eight ticks of a 100 ms timer during one 12-bit conversion, measured.

One honest limitation: a program here **cannot mask interrupts** the way MMBasic does around a byte, and should not be able to. Nothing takes the processor away mid-slot — the kernel cannot be preempted — but a timer interrupt can still stretch one. One-wire tolerates a slot being long rather than short, so a stretched write still reads correctly; if a transfer matters, check the CRC the device gives you.

14.7 Not covered

The editor, `RUN`, `LIST`, `EDIT` and the rest of the immediate-mode environment, which will never apply: a translated program is compiled and run rather than typed at a prompt. (`mmedit` provides the editing they existed for.) The remaining hardware statements are the subject of current work.

Of the graphics, `MODE`, `COLOUR`, `PIXEL` (including the array form), `LINE`, `CIRCLE`, `BOX`, `RBOX`, `TRIANGLE` (its drawing form — `SAVE/RESTORE` need the interpreter's blit buffers), `ARC`, `RGB()`, `FRAMEBUFFER` (including `LAYER` and `MERGE`), `PRINT @`, `TEXT`, `FONT`, `CLS [colour]` and `MAP` (statement and function) are done, and since v0.15 so are `BLIT` — including the flash slots it reads and writes — and the `SPRITE` family with its two-axis `SCROLL`. `TEXT` draws in any of MMBasic's nine built-in fonts, and in a font the program defines for itself with `DefineFont` (numbers 10 to 16), but only in its normal and vertical orientations — the three that rotate the character itself are accepted and drawn normally.

Of the pins, `SETPIN n, DIN|DOUT|AIN|ARAW|INTH|INTL|INTB|PWM|OFF`, `PIN(n) =` and `PIN(n)` are done, and so are `PORT` in both directions and `PULSE`. `PORT(pin, nbits [, pin, nbits]...)` reads or writes several pins as one number, and **the first pin of a group is the least significant bit** — `PORT(0,8) = 1` lights GP0, not GP7. Every pin in a bank changes on the same clock edge, which is the whole reason to use it rather than eight `PIN()` writes: written one at a time, eight lines carry seven wrong values first, and anything clocked off them sees all seven. A port spanning GP31/GP32 takes one store per bank, as it must.

`PULSE pin, width` **inverts** the pin for the width rather than driving it high, which is MMBasic's behaviour. Under 3 ms it blocks and the width is exact; at 3 ms and above it returns at once and the pin flips back later — at the next `PAUSE`, the next `PULSE`, or the next statement in a program that also uses interrupts. MMBasic ends the long ones from a hardware timer, and this machine has no sub-second interval timer to hang one on.

`ONEWIRE RESET`, `WRITE` and `READ` are done, with `MM.ONEWIRE` and MMBasic's four flag bits, and so are `TEMPR` and `TEMPR START` for a DS18B20. `ONEWIRE SEARCH` is not — one device to a pin is what the statements above assume.

Also `PWM slice, freq, duty [, duty2]` with `PWM slice, OFF`, and `SERVO slice, position [, position2]` with `SERVO slice, OFF`; the frequency and counting modes of `SETPIN`, and `PWM SYNC`, are not. Interrupt handlers must be SUBs — MMBasic's label and line-number targets are refused, and with them `IRETURN`, which a SUB handler never needs.

Of the interrupts, `SETPIN INTH|INTL|INTB`, `SETTICK` (all four timers, `PAUSE`, `RESUME`, off) and `ON KEY`

(both forms) are done. They are MMBasic's poll, checked between statements, with its priority order and its no-nesting rule. **A `PAUSE` services them while it waits**, as MMBasic's does — which matters because a program that arms a `SETTICK` usually has a main loop of little else, so a `PAUSE` that ignored it would mean the handler never ran at all. The wait is cut into slices with the poll between them, and the slice is sized from the shortest armed period: a slow tick still sleeps and costs nothing, a fast one spins as MMBasic does, and only while the program is pausing.

Three divergences, all named where they are described: `SETTICK` fires at the period asked for rather than MMBasic's period-plus-a-millisecond; the console is checked at most every 5 ms; and keys reach a program only when a line is complete, which is the tty driver's doing and limits `ON KEY` to command-at-a-time input. `ON PS2`, `INTERRUPT`, `PID` and `SENSORFUSION` have nothing to notify on this machine. The analogue pair need an ADC pin (GP40–GP46 here), and `AIN` uses MMBasic's own ten-sample sort-and-discard filter. `PIN()` returns a float in every mode rather than MMBasic's integer for digital and `ARAW`, and pins are *owned*: `SETPIN` claims from the kernel, only the I/O header may be claimed, and the pin is released and reset when the program ends however it ends. `OPTION VCC` is not supported, so `AIN` always scales by 3.3 V.

Of the sound, `PLAY MP3`, `PLAY WAV`, `PLAY FLAC`, `PLAY VOLUME`, `PLAY STOP`, `PLAY SOUND`, `PLAY TONE`, `PLAY MODFILE` and `PLAY MODSAMPLE` are done — `MIDI`, `LOAD SOUND` and the user-defined `U` waveform are not. `PLAY VOLUME` takes one level rather than one per channel. None of the file players takes the completion interrupt MMBasic allows on them.

Of the statements that reach *into* something rather than replacing it, `MID$(s$,n,m) =`, `LMID(a(),start[,num]) =`, `BIT(v,n) =`, `BYTE(s$,n) =`, `FLAG(n) =` and `FLAGS =` are all done, with `BIT()`, `BYTE()`, `FLAG()` and `MM.INFO(FLAGS)` reading them back. `FLAGS` is sixty-four bits of scratch for the program's own use, cleared at start — and one set *per program* here, where MMBasic has a single firmware global. `LMID` is a **splice, not an overwrite**: `num` bytes come out and the string goes in, so the long string grows or shrinks unless the two lengths match, and leaving `num` out means "as long as the replacement". Where MMBasic checks the target's type when the statement runs, the translator knows an lvalue's type as it generates the call, so a `BIT` on a string is refused at translation with the line named.

`POS` gives the column the next character will go in, 1 at the start of a line. `FLUSH #n` pushes a file out — `fflush` *and* `fsync`, since a program that says `FLUSH` means the version that survives the power going off. Note that this machine's `fsync` syncs the whole filesystem, so it costs more than MMBasic's per-file version; channel 0 is the console and does nothing, as MMBasic's does.

In a graphics mode a program's `PRINT` now draws the characters into whatever is being drawn on, as MMBasic does, so text goes into the framebuffer with everything else. What is still missing is `OPTION CONSOLE`, to say whether a program's output should go to the screen, the serial port, or both.